Numerically stable gradients of Golub-Kahan Bidiagonalization

Theo Würtzen and Nicholas Krämer

October 19, 2026

Abstract

We derive a numerically stable, matrix-free adjoint for Golub-Kahan bidiagonalization. Golub-Kahan bidiagonalization is the standard matrix-free method behind least-squares solvers and partial singular value decompositions. Reverse-mode automatic differentiation stores the Krylov bases of every iteration, so the memory of automatic differentiation grows quadratically with the number of iterations, and without reorthonormalization, automatic differentiation inherits the loss of orthogonality of the forward pass. In our experiments, the adjoint needs about 1.5 times the memory and twice the run time of the forward pass, and computes gradients at least as accurate as automatic differentiation. On a sparse matrix with two million rows, the adjoint differentiates through 100 iterations in 2.6 GB, where automatic differentiation would need 277 GB.

1 Introduction

Fitting the parameters θ of a model often requires the gradient of a function of a large matrix A_θ, that is, of a quantity computed from A_θ by linear algebra. Examples are the solution of a least-squares problem $\min_x \|A_\theta x - b\|_2$, the leading singular values and vectors of A_θ, and the log-determinant $\log\det\big(A_\theta^\top A_\theta\big)$ [6]. Large matrices are often available only *matrix-free*, that is, through a function that computes the products $u \mapsto A_\theta u$ and $w \mapsto A_\theta^\top w$ without forming or storing A_θ in computer memory. Storing all entries of A_θ may be too expensive, as for large sparse or structured matrices, or A_θ may be defined only implicitly, as for convolutions, finite-difference operators, discrete Fourier transforms, or the Jacobian of a neural network. *Krylov methods* compute such quantities matrix-free [4]. For a square matrix A and an initial vector v, a Krylov method builds an orthonormal basis of the Krylov space $\operatorname{span}\{v, Av, \ldots, A^{K-1}v\}$, one basis vector per product with A, and projects A onto this basis. The projection is a small $K \times K$ matrix, from which the method approximates eigenvalues of A, solutions of linear systems with A, or matrix functions of A. The *Arnoldi iteration* orthogonalizes each new vector against all previous basis vectors and computes a Hessenberg factorization $AQ = QH + re_K^\top$, with orthonormal columns Q

and an upper Hessenberg matrix H (Appendix B.1). For a symmetric A, the matrix H is tridiagonal, and the Arnoldi iteration reduces to the *Lanczos iteration*. For a rectangular A, Golub-Kahan bidiagonalization builds one basis for each of the Krylov spaces of $A^\top A$ and $AA^\top$ and reduces A to a bidiagonal matrix.

Golub-Kahan bidiagonalization [3] reduces A to a small bidiagonal matrix B by building orthonormal bases L and R with $AR = LB$, one column per step, from one product with A and one with $A^\top$ (Section 2.2). In floating-point arithmetic, the computed bases lose orthogonality [8], which the forward pass repairs by reorthonormalizing each new vector against the previously computed vectors [4] (Appendix F). Differentiating the iteration is harder. Reverse-mode automatic differentiation through the unrolled loop stores both bases after every step, so the memory of automatic differentiation grows quadratically with the number of steps, and without reorthonormalization, the gradients drift with the loss of orthogonality until the errors of the gradients are as large as the gradients themselves (Figure 1).

The *adjoint method* differentiates an algorithm without storing its intermediate quantities (Appendix A). The adjoint method describes the output of the algorithm by constraints, such as $AQ = QH + re_K^\top$ and $Q^\top Q = I$ for the Arnoldi iteration, and obtains the gradient of a loss from adjoint variables, one per constraint, that solve a linear system, the adjoint system. For Krylov methods, a backward recursion that mirrors the forward iteration solves the adjoint system, so the gradient costs about as much as the forward pass. Krämer et al. [6] derived adjoints of the Arnoldi and Lanczos iterations that are numerically stable and need only matrix-vector products.

This paper derives a stable adjoint for bidiagonalization whose memory grows only linearly with the number of steps. We use that bidiagonalizing A with initial vector v is exactly a Hessenberg factorization of an augmented symmetric matrix built from A, so we can reuse the stable Hessenberg adjoint of Krämer et al. [6]. Most of the augmented problem is exactly zero, and pruning the zeros collapses the adjoint back to the original dimensions. What remains is a small system whose reprojection step does for the backward pass what reorthonormalization does for the forward pass. Section 2 defines bidiagonalization and explains why differentiating through bidiagonalization is difficult, Section 3 states the adjoint as Theorem 1, and Section 4 reports the experiments. The appendix derives the theorem and describes the experimental setups.

2 Golub-Kahan bidiagonalization

2.1 Notation

Let e_j be the jth unit vector. Denote by "$\circ$" the element-wise matrix product which binds weaker than the standard matrix product and stronger than addition, and define the

matrices

$$\begin{aligned} \mathbf{1} &:= [1]_{i,j=1}^{K} & I_{\leq} &:= [\delta_{i\geq j}]_{i,j=1}^{K} & I_{\geq} &:= [\delta_{i\leq j}]_{i,j=1}^{K} \\ I_{<} &:= [\delta_{i>j}]_{i,j=1}^{K} & I_{\ll} &:= [\delta_{i>j+1}]_{i,j=1}^{K} & I_{\geq+1} &:= [\delta_{i\leq j+1}]_{i,j=1}^{K} \end{aligned}$$

so that for example, $\mathbf{1} \circ A = A$, $I_{\leq} \circ A$ extracts the lower triangular matrix of A (including the diagonal), and $I_{\ll} \circ A = 0$ enforces Hessenberg form.

2.2 Definition

Golub-Kahan bidiagonalization [3] reduces a rectangular matrix A to a small bidiagonal matrix, using only matrix-vector products with A and $A^\top$.

Definition 1 (Bidiagonalization)**.**

$$\texttt{Bidiagonalize}_K : (A, v) \mapsto (L, R, B, r, c)$$

Bidiagonalizing a matrix $A \in \mathbb{R}^{N\times M}$ with initial vector $v \in \mathbb{R}^M$ and a prescribed number of iterations $K \in \mathbb{N}_+$ yields column-orthonormal $L \in \mathbb{R}^{N\times K}$ and $R \in \mathbb{R}^{M\times K}$, bidiagonal $B \in \mathbb{R}^{K\times K}$, residual vector $r \in \mathbb{R}^M$, and scalar $c \in \mathbb{R}$, satisfying the following constraints:

$$A^\top L = RB^\top + r(e_K)^\top \tag{1}$$
$$AR = LB \tag{2}$$
$$Re_1 = cv \tag{3}$$
$$I_{\leq} \circ [L^\top L] = I \tag{4}$$
$$I_{\leq} \circ [R^\top R] = I \tag{5}$$
$$R^\top r = 0 \tag{6}$$

and B is full rank and diagonally banded as follows:

$$B = \underbrace{\operatorname{diag}(\boldsymbol{\alpha}) + \operatorname{diag}(\boldsymbol{\beta}, \text{k=1})}_{\text{B is 'upper bidiagonal'}} = \begin{bmatrix} \alpha_1 & \beta_1 & 0 & 0 & 0 & 0 \\ 0 & \alpha_2 & \beta_2 & 0 & 0 & 0 \\ 0 & 0 & \ddots & \ddots & 0 & 0 \\ 0 & 0 & 0 & 0 & \alpha_{K-1} & \beta_{K-1} \\ 0 & 0 & 0 & 0 & 0 & \alpha_K \end{bmatrix} \in \mathbb{R}^{K\times K} \tag{7}$$

The full rank property on B implies that no components of $\boldsymbol{\alpha} \in \mathbb{R}^K$ or $\boldsymbol{\beta} \in \mathbb{R}^{K-1}$ can be zero.

Bidiagonalization is an iterative process: each iteration computes one new column of L from a product with A and one new column of R from a product with $A^\top$, and orthogonalizes each new column only against the previous column of the same basis. In exact arithmetic, the new columns are then orthogonal to all previous columns as well,

so the columns of L and R are orthonormal by construction. In floating-point arithmetic, the rounding errors of each iteration carry over into all later iterations, and the computed columns lose orthogonality as K grows [8]. Implementations therefore *reorthonormalize* every new column against all previous columns of L and R. Appendix F describes the reorthonormalization step, and Section 3 the counterpart of reorthonormalization in the backward pass.

The number of iterations K is limited by the rank of A. Since $B = L^\top AR$ by (2) and (4), B can only be full rank if $K \leq \operatorname{rank}(A)$. More precisely, in exact arithmetic, the iteration yields a full-rank B if and only if K is at most the number of distinct nonzero singular values σ of A for which the projection of v onto the right singular vectors of σ is nonzero. For a generic v, this number is the number of distinct nonzero singular values of A. One more iteration produces a zero component of $\boldsymbol{\alpha}$ or $\boldsymbol{\beta}$, and the iteration breaks down.

2.3 Differentiating through bidiagonalization

Bidiagonalization is usually one step of a larger computation, such as a least-squares solve, a partial singular value decomposition, or a log-determinant estimate, and fitting the parameters of such a computation with gradient-based optimization requires derivatives through the bidiagonalization. We write the rest of the computation as a loss $\rho(L, R, B, r, c)$ of the outputs and seek the gradients of ρ with respect to A_θ, or to the parameters θ, and with respect to v. Reverse-mode automatic differentiation through the K iterations stores the intermediate quantities of every iteration for the backward pass. The reorthonormalization step of every iteration reads the bases L and R, so automatic differentiation stores a copy of both bases, with up to $K(N + M)$ entries, in each of the K iterations. The memory of automatic differentiation therefore grows quadratically in K (Figure 4), whereas the bases themselves need only $K(N + M)$ entries. We instead derive an adjoint for bidiagonalization itself, by way of the Hessenberg factorization. Krämer et al. [6] derived an adjoint for Hessenberg factorization that is numerically stable and needs only matrix-vector products, and Appendix B.2 shows that a Hessenberg factorization of an augmented matrix computes the bidiagonalization, so that the stable adjoint applies to bidiagonalization as well.

3 A stable adjoint for bidiagonalization

Theorem 1 states the adjoint of bidiagonalization. The theorem needs only the outputs of $\texttt{Bidiagonalize}_K$ and the gradients of the loss with respect to these outputs. The appendix derives the theorem in four steps. Appendix B writes bidiagonalization as a Hessenberg factorization of an augmented matrix, Appendix C recalls the stable adjoint system of Hessenberg factorization of Krämer et al. [6], Appendix D shows that most entries of the adjoint variables of the augmented problem are zero, and Appendix E prunes these zeros, which leaves a system in the original dimensions.

Theorem 1 (Stable adjoint of bidiagonalization)**.** For a matrix $A \in \mathbb{R}^{N\times M}$ and an initial vector $v \in \mathbb{R}^M$, let $(L, R, B, r, c) = \texttt{Bidiagonalize}_K(A, v)$, with $B = \operatorname{diag}(\boldsymbol{\alpha}) + \operatorname{diag}(\boldsymbol{\beta}, \mathrm{k}{=}1)$ as in Definition 1, and let $\rho(L, R, B, r, c) \in \mathbb{R}$ be a loss. Write l_n and r_n for the nth columns of L and R, res for the residual r, to distinguish the residual from the columns r_n, ∇_L and ∇_R for $\nabla_L\rho$ and $\nabla_R\rho$, and $\nabla_{\boldsymbol{\alpha}}\rho \in \mathbb{R}^K$ and $\nabla_{\boldsymbol{\beta}}\rho \in \mathbb{R}^{K-1}$ for the gradients with respect to the diagonal and the superdiagonal of B. Let $\Phi = [\phi_1, \ldots, \phi_K] \in \mathbb{R}^{N\times K}$, $\Xi = [\xi_1, \ldots, \xi_K] \in \mathbb{R}^{M\times K}$, lower triangular $\Sigma, \Omega \in \mathbb{R}^{K\times K}$, $\kappa \in \mathbb{R}^M$, and $\eta \in \mathbb{R}^K$ solve the adjoint system

$$\eta = -R^\top \nabla_r\rho + \nabla_{\alpha_K}\rho\, e_K \tag{8}$$
$$\xi_K = \nabla_r\rho + R\eta \tag{9}$$
$$c\nabla_c\rho = r_1^\top \kappa \tag{10}$$
$$0 = \nabla_L + A\Xi - \Phi B^\top + L(\Sigma + \Sigma^\top) \tag{11}$$
$$0 = \nabla_R + A^\top\Phi - \Xi B + R(\Omega + \Omega^\top) + \kappa\, e_1^\top + \operatorname{res}\eta^\top \tag{12}$$
$$I_{\geq} \circ (L^\top\Phi) = \operatorname{diag}(\nabla_{\boldsymbol{\beta}}\rho,\ k{=}1) \tag{13}$$
$$I_{\geq+1} \circ (R^\top\Xi) = \operatorname{diag}(\nabla_{\boldsymbol{\alpha}}\rho) \tag{14}$$

Then the gradients of $\rho \circ \texttt{Bidiagonalize}_K$ with respect to A and v are

$$\nabla_A\rho = \Phi R^\top + L\Xi^\top \tag{15}$$
$$\nabla_v\rho = -c\kappa \tag{16}$$

With $\xi_0 := \kappa$, a backward loop over $n = K, \ldots, 1$ solves the system, like the adjoint of Hessenberg factorization (Appendix C.1), but entirely in the original dimensions. Equations (8) and (9) give ξ_K. In step n, column n of (11) gives ϕ_n, and column n of (12) gives ξ_{n-1}; multiplying these columns by $L^\top$ and $R^\top$ and inserting the reprojection constraints (13) and (14) gives row n of Σ and of Ω.

Parameter gradients. If $A = A(\theta)$ is only available through matrix-vector products, (15) and $\mathrm{d}A = D_\theta A\,\mathrm{d}\theta$ give

$$\nabla_\theta\rho = \sum_{n=1}^{K} \nabla_\theta\big[\phi_n^\top A(\theta)\, r_n + l_n^\top A(\theta)\, \xi_n\big], \tag{17}$$

where ϕ_n, r_n, l_n, and ξ_n are held fixed. Each summand is a vector-Jacobian product of the matrix-vector products $u \mapsto A(\theta)u$ and $w \mapsto A(\theta)^\top w$ that the backward loop evaluates anyway, so the gradient accumulates during the loop, as for the Lanczos and Arnoldi adjoints [6, Corollary 4.3]. Each step of the loop needs one product with A, one with $A^\top$, and inner products with L and R; besides the outputs of the forward pass, the loop stores only the current ϕ_n and ξ_n and the $K \times K$ matrices Ω and Σ.

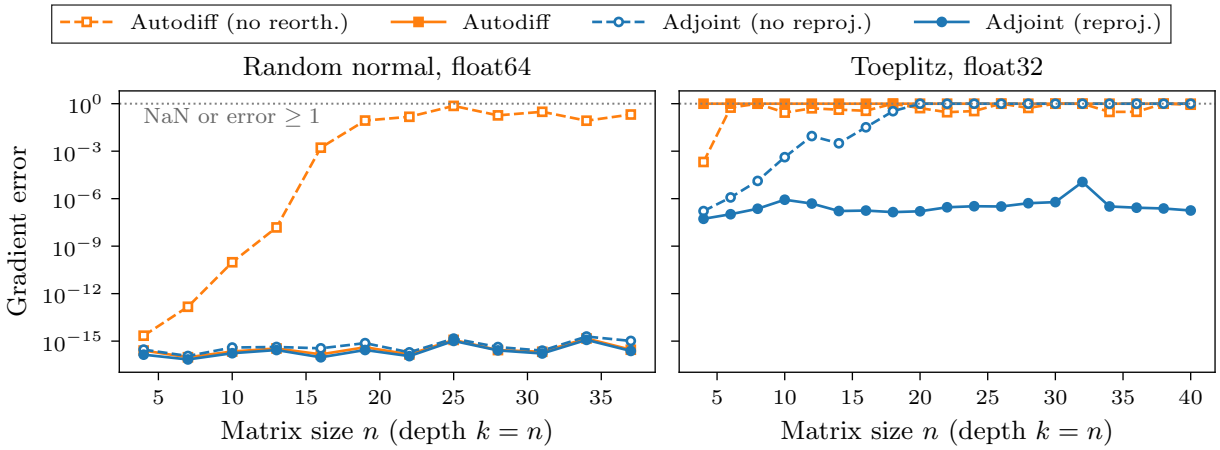


Figure 1: Gradient error at full depth, $\|\partial(LBR^\top)/\partial A - I\|_{\text{rms}}$, against the matrix size n. Left: dense random normal $n \times n$ matrices in double precision. Right: symmetric Toeplitz matrices $C_{ij} = 1/(1 + |i - j|)$ in single precision. NaNs and errors above 1 are drawn at 1. In double precision, autodiff and both variants of the adjoint overlap at machine precision.

Reorthonormalization and reprojection. In floating-point arithmetic, the forward pass loses the orthogonality of L and R and is therefore reorthonormalized against both bases in every step [4]; we use two passes. The reprojection constraints (13)–(14) play the same role in the backward pass: the constraints prescribe $L^\top \phi_n$ and $R^\top \xi_n$, and projecting every new adjoint vector onto these constraints removes the rounding errors that accumulate during the backward loop. Appendix F describes this reprojection step and shows that, in single precision, both corrections are necessary (Figure 5).

4 Experiments

All methods are implemented in JAX [1]. We run the iteration to full depth, where the reconstruction $LBR^\top = A$ provides a ground truth, and to partial depth, the regime of applications. Appendix H describes the setups and reports further results.

4.1 Full depth

At full depth, $k = \min(N, M)$, the map $A \mapsto LBR^\top$ is the identity, and we report the root-mean-square deviation of the computed Jacobian of this map from the identity matrix. We compare automatic differentiation through the iteration (*autodiff*) with the adjoint, with and without reorthonormalization for autodiff and with and without reprojection for the adjoint.

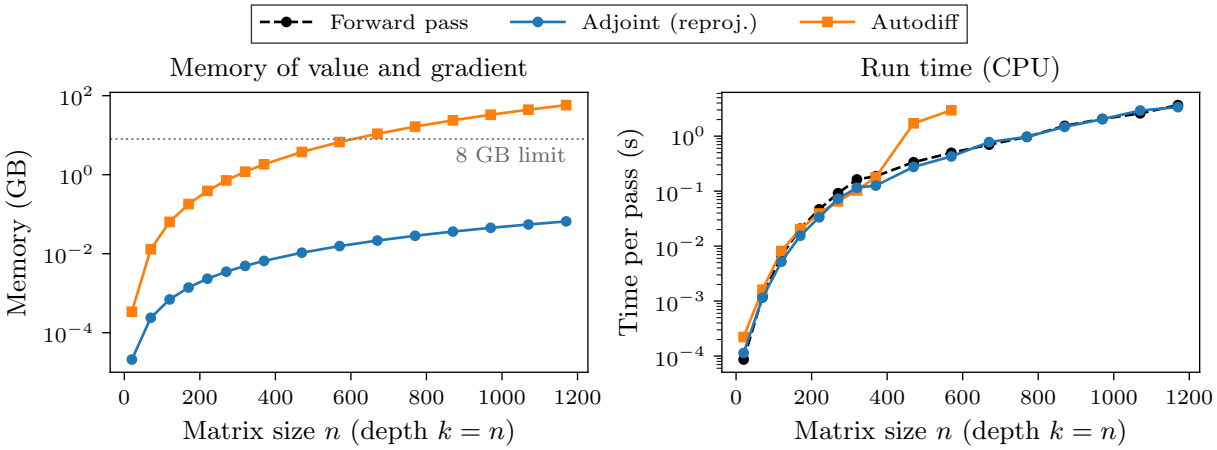


Figure 2: Cost at full depth, $k = n$, for dense random normal $n \times n$ matrices in single precision. Left: memory of the compiled value-and-gradient program, XLA's buffer assignment, which is available without running the program; autodiff was not run above 8 GB. Right: CPU run time of the forward pass, which both methods share, and of each backward pass.

Accuracy. In double precision (Figure 1, left), all methods except autodiff without reorthonormalization reach machine precision. In single precision (right), only the adjoint with reprojection stays accurate, below $1.2 \cdot 10^{-5}$; autodiff through the reorthonormalized iteration returns NaNs, because the last residual is exactly zero at full depth (Appendix H.1).

Cost. The backward pass of the adjoint costs about as much as the forward pass, and the memory of the adjoint grows quadratically in n, like the memory of the forward pass (Figure 2). The memory of autodiff grows cubically, to 430 times the memory of the adjoint at $n = 570$.

4.2 Partial depth

Applications such as LSQR [7] and stochastic Lanczos quadrature [10] use $k \ll \min(N, M)$ steps. We differentiate the k-step quadrature estimate of $v^\top \log\big(A^\top A\big)\, v$,

$$\rho(A) = \|v\|^2\, e_1^\top \log\big(B^\top B\big)\, e_1 = \|v\|^2 \sum_{i=1}^{k} (e_1^\top w_i)^2 \log \sigma_i^2, \tag{18}$$

where $B = \sum_i \sigma_i u_i w_i^\top$ is the singular value decomposition of B; averaged over random start vectors v, ρ estimates $\log\det\big(A^\top A\big)$. Besides the adjoint and autodiff, we compare two alternatives with stable adjoints [6]: the Lanczos adjoint on $A^\top A$, since $B^\top B$ is the tridiagonal matrix of the Lanczos iteration on $A^\top A$, and the Arnoldi adjoint on the augmented matrix of Appendix B.2 (*augmented Arnoldi*). All four methods agree in exact arithmetic (Appendix H.2).

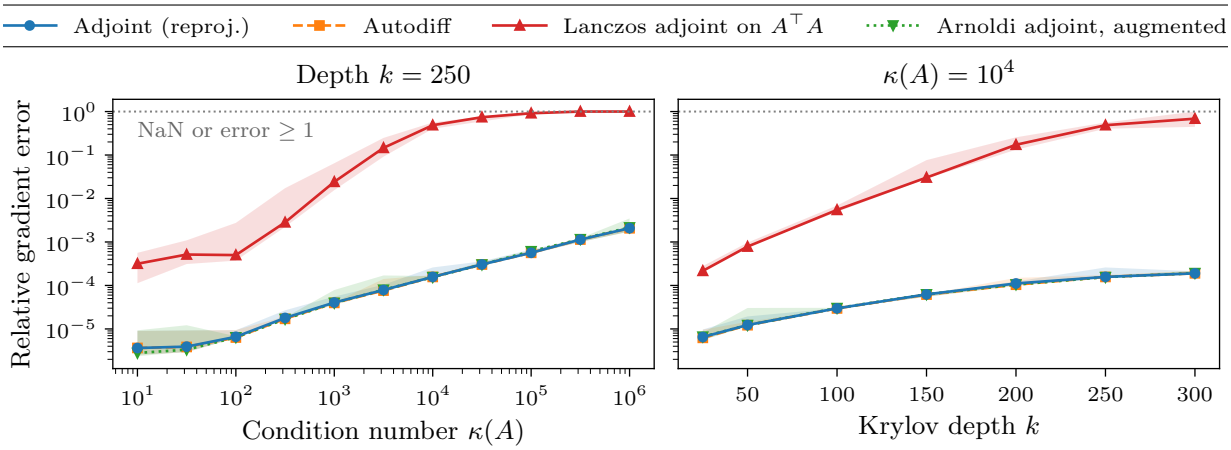


Figure 3: Single-precision gradient error of the log-det quadrature (18) on 1000×500 matrices with singular values decaying geometrically from 1 to $1/\kappa(A)$. Left: against $\kappa(A)$ at depth $k = 250$. Right: against depth at $\kappa(A) = 10^4$. Lines show the median over five random matrices, shaded regions the range; NaNs and errors above 100% are drawn at 1. The adjoint, autodiff, and the augmented formulation overlap, while the Lanczos adjoint on $A^\top A$ squares the condition number and fails.

Accuracy. In single precision (Figure 3), the adjoint, autodiff, and augmented Arnoldi produce the same gradients, with errors from $4 \cdot 10^{-6}$ at $\kappa(A) = 10$ to $2 \cdot 10^{-3}$ at $\kappa(A) = 10^6$. The Lanczos adjoint on $A^\top A$ squares the condition number and reaches 49% error at $\kappa(A) = 10^4$. Since autodiff and augmented Arnoldi are as accurate as the adjoint, the case for the adjoint over these two methods rests on cost.

Memory and run time. For a matrix-free convolution operator with 31 learnable weights and $n = 20{,}000$ (Figure 4), the adjoint needs about $1.5\times$ the memory and 1.6–$2.3\times$ the run time of the forward pass. The memory of autodiff grows quadratically in k, to 190 GB at $k = 512$, where the adjoint needs 0.25 GB. Augmented Arnoldi needs about $4.6\times$ the memory and up to $11\times$ the run time of the adjoint; the Lanczos adjoint costs about as much as the adjoint.

SuiteSparse matrices. On five rectangular SuiteSparse matrices [2, 5] with up to two million rows (Table 1), as in Krämer et al. [6, Figure 3] for the Lanczos iteration, the adjoint needs at most 2.6 GB, where autodiff would need up to 277 GB. Augmented Arnoldi needs 4–$5\times$ the memory of the adjoint. On tall matrices, the Lanczos adjoint is up to $3\times$ faster than the adjoint, because the Lanczos iteration orthogonalizes against one basis instead of two, but the Lanczos adjoint is 17–$32\times$ less accurate on three of the five matrices.

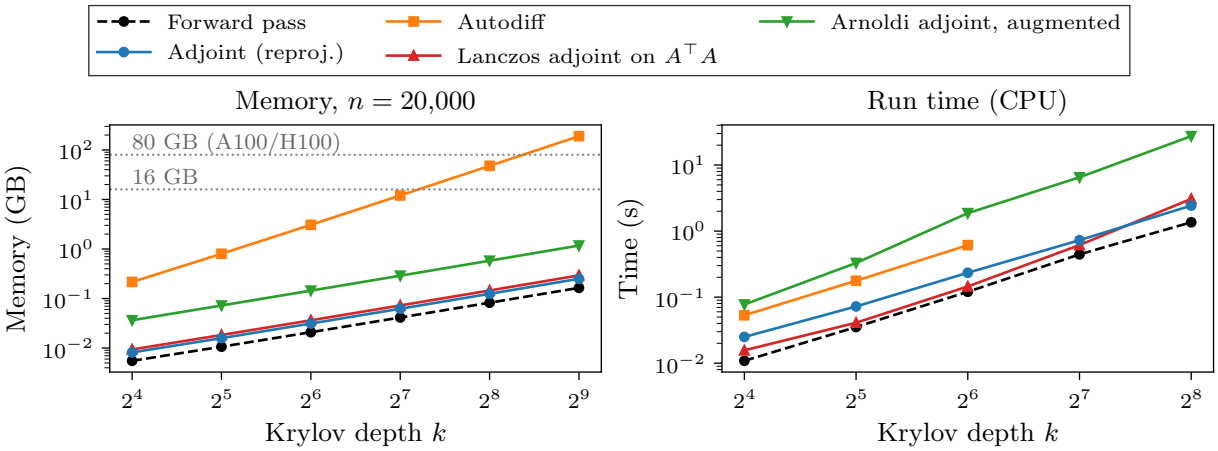


Figure 4: Memory (left) and CPU run time (right) of the value and gradient of the log-det quadrature (18) for a matrix-free convolution operator with $n = 20{,}000$, against depth k. Memory is XLA's compiled buffer assignment, which is available without running the program; programs above 12 GB were not timed. Dotted lines mark common accelerator memory sizes.

Matrix	Method	Memory (GB)	Time (s)	Gradient error
`Kemelmacher`	Forward pass	0.03	0.27	
$28{,}452 \times 9{,}693$	**Adjoint (reproj.)**	0.05	0.50	$1.4 \cdot 10^{-6}$
0.10M nonzeros	Autodiff	5.97	2.88	$1.4 \cdot 10^{-6}$
	Lanczos adjoint on $A^\top A$	0.03	0.26	$2.4 \cdot 10^{-5}$
	Arnoldi adjoint, augmented	0.22	2.91	$1.7 \cdot 10^{-6}$
`tomographic1`	Forward pass	0.11	1.00	
$73{,}159 \times 59{,}498$	**Adjoint (reproj.)**	0.17	2.22	$7.8 \cdot 10^{-6}$
0.65M nonzeros	Autodiff	24.2	—	—
	Lanczos adjoint on $A^\top A$	0.18	2.08	$2.5 \cdot 10^{-4}$
	Arnoldi adjoint, augmented	0.75	16.31	$8.3 \cdot 10^{-6}$
`image_interp`	Forward pass	0.30	2.80	
$240{,}000 \times 120{,}000$	**Adjoint (reproj.)**	0.44	5.82	$7.7 \cdot 10^{-6}$
0.71M nonzeros	Autodiff	59.6	—	—
	Lanczos adjoint on $A^\top A$	0.35	4.39	$4.6 \cdot 10^{-6}$
	Arnoldi adjoint, augmented	2.03	43.79	$6.5 \cdot 10^{-6}$
`ESOC`	Forward pass	0.34	5.50	
$327{,}062 \times 37{,}830$	**Adjoint (reproj.)**	0.52	11.70	$6.3 \cdot 10^{-4}$
6.02M nonzeros	Autodiff	54.5	—	—
	Lanczos adjoint on $A^\top A$	0.22	8.09	$1.4 \cdot 10^{-2}$
	Arnoldi adjoint, augmented	2.10	51.34	$1.5 \cdot 10^{-3}$
`Rucci1`	Forward pass	1.74	18.07	
$1{,}977{,}885 \times 109{,}900$	**Adjoint (reproj.)**	2.60	36.50	$5.4 \cdot 10^{-4}$
7.79M nonzeros	Autodiff	277.0	—	—
	Lanczos adjoint on $A^\top A$	0.45	11.79	$5.7 \cdot 10^{-4}$
	Arnoldi adjoint, augmented	11.8	—	—

Table 1: Value and gradient of the log-det quadrature (18) with respect to the nonzero values of five rectangular SuiteSparse matrices, at depth $k = 100$ in single precision. Memory is XLA's compiled buffer assignment; programs above 8 GB were not run, and dashes mark the missing entries. Gradient errors are relative to a double-precision reference. Setup in Appendix H.2.

5 Conclusion

By viewing bidiagonalization as a Hessenberg factorization of an augmented symmetric matrix, we obtained a stable adjoint for bidiagonalization almost for free, and pruning the zeros of the augmented system brought the cost back down to the cost of the original problem. The adjoint needs nothing beyond matrix-vector products and a reprojection step. In our experiments, the adjoint computed gradients as accurate as automatic differentiation through the reorthonormalized iteration, at a fraction of the memory, and the reprojection step was necessary to keep the gradients accurate in single precision. At partial depth, the adjoint was the only method we tested that combines the accuracy of differentiating through the bidiagonalization with memory linear in the depth: autodiff and the augmented formulation are accurate but expensive, and the Lanczos adjoint on $A^\top A$ is cheap but squares the condition number. On tall matrices, the Lanczos adjoint is also faster, because the Lanczos iteration orthogonalizes only against the shorter of the two bases; one-sided reorthogonalization [9] might close this gap for bidiagonalization as well. The same augment-then-prune recipe should carry over to other structured factorizations that embed into a Hessenberg or symmetric Lanczos form. We leave both directions to future work.

References

[1] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL `http://github.com/jax-ml/jax`.

[2] Timothy A. Davis and Yifan Hu. The University of Florida sparse matrix collection. *ACM Transactions on Mathematical Software*, 38(1):1–25, 2011. doi: 10.1145/2049662.2049663.

[3] Gene Golub and William Kahan. Calculating the singular values and pseudo-inverse of a matrix. *Journal of the Society for Industrial and Applied Mathematics, Series B: Numerical Analysis*, 2(2):205–224, 1965. doi: 10.1137/0702016.

[4] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD, 4th edition, 2013.

[5] Scott P. Kolodziej, Mohsen Aznaveh, Matthew Bullock, Jarrett David, Timothy A. Davis, Matthew Henderson, Yifan Hu, and Read Sandstrom. The SuiteSparse matrix collection website interface. *Journal of Open Source Software*, 4(35):1244, 2019. doi: 10.21105/joss.01244.

[6] Nicholas Krämer, Pablo Moreno-Muñoz, Hrittik Roy, and Søren Hauberg. Gradients

of functions of large matrices. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL `https://arxiv.org/abs/2405.17277`. arXiv:2405.17277.

[7] Christopher C. Paige and Michael A. Saunders. LSQR: An algorithm for sparse linear equations and sparse least squares. *ACM Transactions on Mathematical Software*, 8 (1):43–71, 1982. doi: 10.1145/355984.355989.

[8] Christopher Conway Paige. *The Computation of Eigenvalues and Eigenvectors of Very Large Sparse Matrices*. PhD thesis, University of London, 1971.

[9] Horst D. Simon and Hongyuan Zha. Low-rank matrix approximation using the Lanczos bidiagonalization process with applications. *SIAM Journal on Scientific Computing*, 21(6):2257–2274, 2000.

[10] Shashanka Ubaru, Jie Chen, and Yousef Saad. Fast estimation of $tr(f(a))$ via stochastic Lanczos quadrature. *SIAM Journal on Matrix Analysis and Applications*, 38(4): 1075–1099, 2017. doi: 10.1137/16M1104974.

Appendix

The appendix derives Theorem 1 and describes the experiments. Appendix A recalls the adjoint method. Appendix B defines Hessenberg factorization and shows that a Hessenberg factorization of an augmented matrix computes a bidiagonalization. Appendix C states the adjoint system of Hessenberg factorization and recalls how a backward recursion solves it. Appendix D shows that most entries of the adjoint variables of the augmented problem are zero, and Appendix E prunes these zeros, which gives the system of Theorem 1. Appendix F explains reorthonormalization and reprojection, Appendix G states the adjoint system column by column, and Appendix H describes the experimental setups and reports further results.

A The adjoint method

The adjoint method computes parameter gradients of losses that depend on the output of a parameterized numerical solver. Let $s : \theta \mapsto x$ be a solver that yields a solution x to a problem θ, with $\dim(\theta) = P$ and $\dim(x) = X$. Suppose we know a constraint function $g : (x, \theta) \mapsto g(x, \theta) \in \mathbb{R}^X$ with $g(s(\theta), \theta) = \mathbf{0}$ for all θ and with $\frac{\partial g}{\partial x}$ everywhere nonsingular, and a loss $\ell : x \mapsto \ell(x) \in \mathbb{R}$. We seek $\nabla_\theta \ell(s(\theta))$.

Example 1 (Hessenberg factorization)**.** *For $s =$ `Hessenberg` we have $\theta = (A, v)$ and $x = (Q, H, r, c)$. The constraint function is the concatenation of constraints* (22) *through* (26)*, so $g(x, \theta) = \mathbf{0}$ exactly when (Q, H, r, c) correctly Hessenberg-factorizes (A, v).*

Because $g(s(\theta), \theta) = \mathbf{0}$ is constantly zero, the total derivative of $g(s(\theta), \theta)$ is also zero, so the chain rule gives

$$\mathbf{0} = \frac{d}{d\theta} g(s(\theta), \theta) = \frac{\partial g}{\partial x}\frac{\partial s}{\partial \theta} + \frac{\partial g}{\partial \theta} \implies \frac{\partial g}{\partial x}\frac{\partial s}{\partial \theta} = -\frac{\partial g}{\partial \theta} \tag{19}$$

Applying the chain rule to the gradient itself,

$$\nabla_\theta \ell(s(\theta)) = \frac{\partial s}{\partial \theta}^\top \nabla_x \ell(s(\theta)) \tag{20}$$

we assume $\nabla_x \ell(s(\theta))$ is easy to obtain. The challenge is $\frac{\partial s}{\partial \theta} \in \mathbb{R}^{X \times P}$, which is expensive to compute when X and P are large. If we can find *adjoint variables* $\boldsymbol{\lambda} \in \mathbb{R}^X$ solving the uniquely determined $X \times X$ linear system

$$\nabla_x \ell(s(\theta)) = \frac{\partial g}{\partial x}^\top \boldsymbol{\lambda} \tag{21}$$

then substituting into (20) yields

$$\nabla_\theta \ell(s(\theta)) = \frac{\partial s}{\partial \theta}^\top \frac{\partial g}{\partial x}^\top \boldsymbol{\lambda} = \left(\frac{\partial g}{\partial x}\frac{\partial s}{\partial \theta}\right)^\top \boldsymbol{\lambda} \overset{(19)}{=} -\frac{\partial g}{\partial \theta}^\top \boldsymbol{\lambda}$$

This adjoint formula is much cheaper than the direct approach when there are many parameters. Computing $\frac{\partial s}{\partial \theta}$ directly means solving (19), a linear system with the P columns of $-\frac{\partial g}{\partial \theta}$ as right-hand sides, whereas the adjoint approach solves (21), a linear system with a single right-hand side, and then evaluates the product $-\frac{\partial g}{\partial \theta}^\top \boldsymbol{\lambda}$. For bidiagonalization, the parameters are the $NM + M$ entries of A and v, so the direct approach would solve $NM + M$ linear systems where the adjoint approach solves one.

B Hessenberg factorization and augmented bidiagonalization

B.1 Hessenberg factorization

Appendix B.2 computes bidiagonalization with a Hessenberg factorization, the counterpart of bidiagonalization for square matrices, which reduces a square matrix A to a small upper Hessenberg matrix H.

Definition 2 (Hessenberg factorization)**.**

$$\texttt{Hessenberg}_K : (A, v) \mapsto (Q, H, r, c)$$

Given a square matrix $A \in \mathbb{R}^{N\times N}$, an initial vector $v \in \mathbb{R}^N$, and a prescribed number of iterations $K \in \mathbb{N}_+$, the Hessenberg factorization produces a column-orthonormal $Q \in \mathbb{R}^{N\times K}$, upper Hessenberg $H \in \mathbb{R}^{K\times K}$, residual vector $r \in \mathbb{R}^N$, and scalar $c \in \mathbb{R}$ such that

$$AQ = QH + r(e_K)^\top \tag{22}$$
$$Qe_1 = cv \tag{23}$$
$$I_{\leq} \circ [Q^\top Q] = I \tag{24}$$
$$I_{\ll} \circ H = 0 \tag{25}$$
$$Q^\top r = 0 \tag{26}$$

The Arnoldi iteration computes the Hessenberg factorization matrix-free, one column of Q per iteration; the experiments in Section 4 refer to the Hessenberg factorization by the name of this algorithm.

B.2 Using Hessenberg factorization to compute bidiagonalization

We can implement $\texttt{Bidiagonalize}_K(A, v)$ by passing an augmented version of (A, v) through $\texttt{Hessenberg}_{2K}$. For $A \in \mathbb{R}^{N\times M}$ and $v \in \mathbb{R}^M$, we construct the symmetric matrix $\ddot{\ddot{A}}$ and augmented vector $\ddot{\ddot{v}}$:

$$\ddot{\ddot{A}} := \begin{bmatrix} \mathbf{0} & A \\ A^\top & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{(N+M)\times(N+M)} \tag{27}$$

$$\dddot{v} := \begin{pmatrix} \mathbf{0} \\ v \end{pmatrix} \in \mathbb{R}^{N+M} \tag{28}$$

We define the operation $\texttt{augment} : (A, v) \mapsto (\dddot{A}, \ddot{v})$ as the augmentation of a matrix and vector to this symmetric matrix and padded vector. Applying Hessenberg factorization for $2K$ iterations to this augmented input yields the output $\texttt{Hessenberg}_{2K}(\dddot{A}, \ddot{v}) = (\dddot{Q}, \dddot{H}, \ddot{r}, c)$, which is directly related to the output (L, R, B, r, c) of bidiagonalization through the following sparse equations:

$$\dddot{Q} := \begin{bmatrix} \mathbf{0} & l_1 & \mathbf{0} & l_2 & \cdots & \mathbf{0} & l_K \\ r_1 & \mathbf{0} & r_2 & \mathbf{0} & \cdots & r_K & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{(N+M)\times 2K} \tag{29}$$

$$\ddot{r} := \begin{pmatrix} \mathbf{0} \\ r \end{pmatrix} \in \mathbb{R}^{N+M} \tag{30}$$

$$\dddot{H} := \begin{bmatrix} 0 & \alpha_1 & 0 & 0 & 0 & 0 & 0 & 0 \\ \alpha_1 & 0 & \beta_1 & 0 & 0 & 0 & 0 & 0 \\ 0 & \beta_1 & 0 & \alpha_2 & 0 & 0 & 0 & 0 \\ 0 & 0 & \alpha_2 & 0 & \beta_2 & 0 & 0 & 0 \\ 0 & 0 & 0 & \beta_2 & 0 & \ddots & 0 & 0 \\ 0 & 0 & 0 & 0 & \ddots & 0 & \beta_{K-1} & 0 \\ 0 & 0 & 0 & 0 & 0 & \beta_{K-1} & 0 & \alpha_K \\ 0 & 0 & 0 & 0 & 0 & 0 & \alpha_K & 0 \end{bmatrix} \in \mathbb{R}^{2K\times 2K} \tag{31}$$

The vectors l_i and r_i in $\dddot{Q}$ are exactly the columns of L and R. $\dddot{H}$ is symmetric and banded, where the two bands hold the entries of the bidiagonal matrix B with twice repeated interwoven $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ values. Recall that $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ were everywhere nonzero. The residual vector $\ddot{r}$ is a padded version of r, and the scalar c is unchanged.

Temporary: naming options for the augmented quantities. Each row shows a matrix, a vector, a column, a gradient subscript, a transpose, and an adjoint variable of the augmented problem.

triple dot (current)	$\dddot{A}$	$\ddot{v}$	$\ddot{q}_i$	$\nabla_{\ddot{H}}\rho$	$\dddot{Q}^\top$	$\dddot{\Lambda}$
tilde	$\tilde{A}$	$\tilde{v}$	$\tilde{q}_i$	$\nabla_{\tilde{H}}\rho$	$\tilde{Q}^\top$	$\tilde{\Lambda}$
breve	$\breve{A}$	$\breve{v}$	$\breve{q}_i$	$\nabla_{\breve{H}}\rho$	$\breve{Q}^\top$	$\breve{\Lambda}$
ring	$\mathring{A}$	$\mathring{v}$	$\mathring{q}_i$	$\nabla_{\mathring{H}}\rho$	$\mathring{Q}^\top$	$\mathring{\Lambda}$
bar	$\bar{A}$	$\bar{v}$	$\bar{q}_i$	$\nabla_{\bar{H}}\rho$	$\bar{Q}^\top$	$\bar{\Lambda}$
sans-serif	A	v	q_i	$\nabla_{\mathsf{H}}\rho$	$\mathsf{Q}^\top$	$\mathsf{\Lambda}$
blackboard	$\mathbb{A}$	$\mathbb{v}$	$\mathbb{q}_i$	$\nabla_{\mathbb{H}}\rho$	$\mathbb{Q}^\top$	(none)
superscript	A^{aug}	v^{aug}	q_i^{aug}	$\nabla_{H^{\text{aug}}}\rho$	$(Q^{\text{aug}})^\top$	Λ^{aug}

We formalize this relationship through a linear extraction mapping

$$\texttt{extract} : (\dddot{Q}, \dddot{H}, \ddot{r}, c) \mapsto (L, R, B, r, c)$$

that just projects away the zeros and extracts the relevant non-zero entries from the Hessenberg outputs.[1] With this mapping, we formalize **augmented bidiagonalization** as

$$\overline{\texttt{Bidiagonalize}}_K := \texttt{extract} \circ \texttt{Hessenberg}_{2K} \circ \texttt{augment} : (A, v) \mapsto (L, R, B, r, c)$$

which is functionally equivalent to standard bidiagonalization but inherits the numerical stability properties of Hessenberg factorization's adjoint system.
The augmented problem has dimension $N+M$ and runs for $2K$ iterations, but most entries of $\dddot{A}$, $\dddot{Q}$, and $\dddot{H}$ are exactly zero. Appendices D and E prune these zeros from the adjoint system of the augmented problem, which gives Theorem 1.

C The adjoint system of Hessenberg factorization

Krämer et al. [6, Theorem 4.1] derive the adjoint system of Hessenberg factorization, which Proposition 1 restates.

Proposition 1 (Adjoint system of Hessenberg factorization)**.** Let $K \in \mathbb{N}_+$, $A \in \mathbb{R}^{N\times N}$, $v \in \mathbb{R}^N$, $H \in \mathbb{R}^{K\times K}$, $Q \in \mathbb{R}^{N\times K}$, $r \in \mathbb{R}^N$, $c \in \mathbb{R}$ satisfy the primal constraints (22) through (26), and let $\rho(Q, H, r, c) \in \mathbb{R}$ be a loss. Let adjoint variables $\Lambda \in \mathbb{R}^{N\times K}$ with $\Lambda e_i = \lambda_i$, $\Gamma, \mathcal{S} \in \mathbb{R}^{K\times K}$, $\lambda_0 \in \mathbb{R}^N$, $\gamma \in \mathbb{R}^K$ solve the following system of equations:

$$0 = \nabla_Q \rho + A^\top \Lambda - \Lambda H^\top + \lambda_0 (e_1)^\top + Q(I_{\leq} \circ \Gamma) + Q(I_{\leq} \circ \Gamma)^\top + r\gamma^\top \tag{32}$$

$$0 = \nabla_H \rho - Q^\top \Lambda + I_{\ll} \circ \mathcal{S} \tag{33}$$

$$0 = \nabla_r \rho - \Lambda e_K + Q\gamma \tag{34}$$

$$0 = \nabla_c \rho - v^\top \lambda_0 \tag{35}$$

The gradients $\nabla_A \rho$ and $\nabla_v \rho$ are then given by[2]:

$$\nabla_A \rho = \Lambda Q^\top \tag{36}$$

$$\nabla_v \rho = -c\lambda_0 \tag{37}$$

Appendix C.1 recalls how to solve this system with a backward recursion over the columns of Λ and Γ: the initialization (41) and (38), followed by the steps (44) and (45) for $i = K, \ldots, 1$. Each step needs one product with $A^\top$, so the backward pass costs as much as the forward pass.

[1] The projection from $\dddot{H}$ to B is not unique, as $\dddot{H}$ contains two copies of each α_i and β_i. We arbitrarily select the superdiagonal entries and drop the subdiagonal entries. This choice affects the sparsity pattern of $\nabla_{\dddot{H}} \rho$ in Appendix D.1. Any other linear combination of each pair whose two weights sum to one would also be valid, because the two entries of each pair, and the derivatives of the two entries, are equal for every augmented input.

[2] Note a sign difference between (37) and the corresponding result of Krämer et al. [6], which is due to a typo there.

C.1 Solving the adjoint system with a backward recursion

This subsection recalls how to solve the adjoint system of Proposition 1 with a backward recursion, following Krämer et al. [6]. The output gradients $\nabla_Q \rho$, $\nabla_H \rho$, $\nabla_r \rho$, $\nabla_c \rho$ are known, so we need only determine the adjoint variables Λ, Γ, λ_0, γ. Γ is involved only in equation (32) in which only the lower triangular part is accessed, so for notational convenience we will redefine Γ to be lower triangular, meaning we can effectively drop the mask $\Gamma := I_{\leq} \circ \Gamma$.
We get expressions for λ_K and γ:

$$\begin{aligned}
(34) \iff & \lambda_K = \nabla_r \rho + Q\gamma && (38)\\
\overset{(24)}{\implies} & Q^\top \lambda_K = Q^\top \nabla_r \rho + Q^\top Q \gamma \\
& \quad = Q^\top \nabla_r \rho + \gamma \\
\iff & \gamma = Q^\top \lambda_K - Q^\top \nabla_r \rho && (39)
\end{aligned}$$

To evaluate $Q^\top \lambda_K$, inspect (33):

$$\begin{aligned}
(33) \iff & Q^\top \Lambda = \nabla_H \rho + I_{\ll} \circ \mathcal{S} \\
\implies & (\mathbf{1} - I_{\ll}) \circ Q^\top \Lambda = (\mathbf{1} - I_{\ll}) \circ \nabla_H \rho + (\mathbf{1} - I_{\ll}) \circ I_{\ll} \circ \mathcal{S} \\
& \quad = (\mathbf{1} - I_{\ll}) \circ \nabla_H \rho && (40)\\
\implies & Q^\top \lambda_K = \nabla_H \rho e_K \\
\overset{(39)}{\implies} & \gamma = \nabla_H \rho e_K - Q^\top \nabla_r \rho && (41)
\end{aligned}$$

This final relation lets us express γ in terms of the known quantities, which we can then plug into (38) to get λ_K.
Next, we will get expressions for the remaining columns of Λ and Γ, as well as for λ_0. As a preliminary, from primal constraints (23) we have $v = q_1/c$, yielding

$$(35) \iff c\nabla_c \rho = q_1^\top \lambda_0 \tag{42}$$

We now turn to adjoint system constraint (32), left-multiply $Q^\top$ and then apply a mask to consider only the upper triangular part of the equations:

$$\begin{aligned}
(32) \overset{(24),(26)}{\implies} & \mathbf{0} = Q^\top \nabla_Q \rho + Q^\top A^\top \Lambda - Q^\top \Lambda H^\top + Q^\top \lambda_0 (e_1)^\top + \Gamma + \Gamma^\top \\
\implies & \mathbf{0} = I_{\geq} \circ \left[Q^\top \nabla_Q \rho + Q^\top A^\top \Lambda - Q^\top \Lambda H^\top + Q^\top \lambda_0 (e_1)^\top + \Gamma + \Gamma^\top \right] \\
& \quad = I_{\geq} \circ \left[Q^\top \nabla_Q \rho + Q^\top A^\top \Lambda - Q^\top \Lambda H^\top + Q^\top \lambda_0 (e_1)^\top \right] + \operatorname{diag}(\Gamma) + \Gamma^\top
\end{aligned}$$

We now inspect column i in this matrix equation. We will use sparsity of H to rewrite the matrix product $Q^\top \Lambda H^\top$, and we will apply some simplifications with the mask $I_\geq$:

$$\begin{aligned}
\mathbf{0} &= \left(I_\geq \circ \left[Q^\top \nabla_Q \rho + Q^\top A^\top \Lambda - Q^\top \Lambda H^\top + Q^\top \lambda_0 (e_1)^\top \right] \right) e_i + \left(\operatorname{diag}(\Gamma) + \Gamma^\top \right) e_i \\
&= I_\geq e_i \circ \left[Q^\top \nabla_{q_i} \rho + Q^\top A^\top \lambda_i - Q^\top \Lambda H^\top e_i + Q^\top \lambda_0 (e_1)^\top e_i \right] + \operatorname{diag}(\Gamma) e_i + \Gamma^\top e_i \\
&= I_\geq e_i \circ \left[Q^\top \nabla_{q_i} \rho + Q^\top A^\top \lambda_i - Q^\top \left(\sum_{j=i-1}^{K} \lambda_j h_{ij} \right) \right] + (I_\geq e_i) \circ Q^\top \lambda_0 \delta_{i=1} + \Gamma_{ii} e_i + \Gamma^\top e_i \\
&= I_\geq e_i \circ \left[Q^\top \nabla_{q_i} \rho + Q^\top A^\top \lambda_i - \sum_{j=i-1}^{K} h_{ij} Q^\top \lambda_j \right] + e_1 q_1^\top \lambda_0 \delta_{i=1} + \Gamma_{ii} e_i + \Gamma^\top e_i \\
&\overset{(42)}{=} I_\geq e_i \circ \left[Q^\top \nabla_{q_i} \rho + Q^\top A^\top \lambda_i \right] - I_\geq e_i \circ \left(\sum_{j=i-1}^{K} h_{ij} Q^\top \lambda_j \right) + e_1 c \nabla_c \rho \delta_{i=1} + \Gamma_{ii} e_i + \Gamma^\top e_i \\
&= I_\geq e_i \circ \left[Q^\top \nabla_{q_i} \rho + Q^\top A^\top \lambda_i \right] - \left(\sum_{j=i-1}^{K} h_{ij} I_\geq e_i \circ Q^\top \lambda_j \right) + e_1 c \nabla_c \rho \delta_{i=1} + \Gamma_{ii} e_i + \Gamma^\top e_i
\end{aligned} \tag{43}$$

We can use (40) to rewrite the term in round parentheses. Column j of the mask $\mathbf{1} - I_{\ll}$ keeps the rows $1, \ldots, j+1$, which include the rows $1, \ldots, i$ that $I_\geq e_i$ keeps whenever $j \geq i-1$. Hence (40) implies

$$I_\geq e_i \circ Q^\top \lambda_j = I_\geq e_i \circ \nabla_H \rho \, e_j \qquad \text{for all } j \geq i-1,$$

where the case $j = i-1$ uses the first subdiagonal of (40). We plug this into (43), which then yields an expression for column i of $\Gamma^\top$ that requires, besides known quantities, only λ_i:

$$\Gamma_{ii} e_i + \Gamma^\top e_i = -I_\geq e_i \circ \left[Q^\top \nabla_{q_i} \rho + Q^\top A^\top \lambda_i \right] + \left(\sum_{j=i-1}^{K} h_{ij} I_\geq e_i \circ \nabla_H \rho e_j \right) - e_1 c \nabla_c \rho \delta_{i=1} \tag{44}$$

Finally, to solve for λ_{i-1} and λ_0, we look again at (32) and restrict to column i:

$$\begin{aligned}
(32) \implies \qquad & 0 = \nabla_{q_i} \rho + A^\top \lambda_i - \Lambda H^\top e_i + \lambda_0 \delta_{i=1} + Q \Gamma e_i + Q \Gamma^\top e_i + r \gamma^\top e_i \\
& = \nabla_{q_i} \rho + A^\top \lambda_i - \sum_{j=i-1}^{K} h_{ij} \lambda_j + \lambda_0 \delta_{i=1} + Q(\Gamma + \Gamma^\top) e_i + r \gamma^\top e_i \\
\iff \quad \begin{matrix} -\lambda_0 & i = 1 \\ h_{i,i-1} \lambda_{i-1} & i > 1 \end{matrix} \Bigg\} & = \nabla_{q_i} \rho + A^\top \lambda_i - \sum_{j=i}^{K} h_{ij} \lambda_j + Q(\Gamma + \Gamma^\top) e_i + r \gamma^\top e_i
\end{aligned} \tag{45}$$

We now have all equations necessary to solve for all adjoint variables. There is a part that we have not yet addressed: In (45), we will need access to $(\Gamma + \Gamma^\top)e_i$, however (44) will only give us $\Gamma^\top e_i$. How we obtain $(\Gamma + \Gamma^\top)e_i$ from the columns of $\Gamma^\top$ is best explained with an example.

Example 2 (Construction of $\Gamma + \Gamma^\top$)**.** *Let $K = 4$. For decreasing i starting from K down to 1, the known entries in upper triangular $\Gamma^\top \in \mathbb{R}^{4\times 4}$ will evolve as follows, where ? denotes unknown entries, and 0 and $\cdot$ denotes known entries:*

$$\Gamma^\top: \quad \underset{\begin{bmatrix} ? & ? & ? & \cdot \\ 0 & ? & ? & \cdot \\ 0 & 0 & ? & \cdot \\ 0 & 0 & 0 & \cdot \end{bmatrix}}{i=4} \qquad \underset{\begin{bmatrix} ? & ? & \cdot & \cdot \\ 0 & ? & \cdot & \cdot \\ 0 & 0 & \cdot & \cdot \\ 0 & 0 & 0 & \cdot \end{bmatrix}}{i=3} \qquad \underset{\begin{bmatrix} ? & \cdot & \cdot & \cdot \\ 0 & \cdot & \cdot & \cdot \\ 0 & 0 & \cdot & \cdot \\ 0 & 0 & 0 & \cdot \end{bmatrix}}{i=2} \qquad \underset{\begin{bmatrix} \cdot & \cdot & \cdot & \cdot \\ 0 & \cdot & \cdot & \cdot \\ 0 & 0 & \cdot & \cdot \\ 0 & 0 & 0 & \cdot \end{bmatrix}}{i=1}$$

These known entries are enough to determine the ith column of symmetric $\Gamma + \Gamma^\top$, whose entries we will uncover in the following sequence:

$$\Gamma + \Gamma^\top: \quad \underset{\begin{bmatrix} ? & ? & ? & \cdot \\ ? & ? & ? & \cdot \\ ? & ? & ? & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix}}{i=4} \qquad \underset{\begin{bmatrix} ? & ? & \cdot & \cdot \\ ? & ? & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix}}{i=3} \qquad \underset{\begin{bmatrix} ? & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix}}{i=2} \qquad \underset{\begin{bmatrix} \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix}}{i=1}$$

We now give the order in which to apply all the equations:
Initially, use (41) to get γ, then follow that up with (38) to get λ_K.
Loop for $i = K$ down to 1:
Use (44) to get $\Gamma^\top e_i$ to then get $(\Gamma + \Gamma^\top)e_i$. Use in (45) to solve for λ_{i-1}, or, if $i = 1$, λ_0.
Finally, we know the adjoint variables and can compute the gradients in (36) and (37).

D Sparsity of the augmented adjoint system

As described in Appendix B.2, augmented bidiagonalization $\overline{\texttt{Bidiagonalize}}_K$ computes bidiagonalization through $\texttt{Hessenberg}_{2K}$, so the adjoint system of Proposition 1 applies to augmented bidiagonalization. The adjoint variables $\Lambda, \Gamma, \lambda_0, \gamma$ have $N + M$ rows or $2K$ columns, but most entries of the adjoint variables are zero, because the augmented inputs $\dddot{A}$, $\dddot{v}$ and outputs $\dddot{Q}, \dddot{H}$, $\dddot{r}$ have the alternating block structure of (27)–(31). Proposition 2 makes this statement precise, using a notation for sparsity patterns that we introduce first.

Sparsity notation. We characterize the sparsity and dimensionality of vectors and matrices succinctly as follows. We use the symbol $*$ to denote an unknown scalar. Thus,

$(*, 0, 0)^\top$ can be thought of as the span of the first unit vector. Formally, define an extension of the reals as $\mathbb{R}_* = \mathbb{R} \cup \{*\}$, where the sum and product with $*$ are extended as

$$s + * = * \qquad s \cdot * = \begin{cases} 0 & s = 0 \\ * & \text{otherwise} \end{cases}$$

These rules imply $* - * = *$. Now, for $s, s' \in \mathbb{R}_*$, define partial order

$$s \preceq s' \iff \square(s) \subseteq \square(s')$$

where

$$\square(s) = \begin{cases} \{s\} & s \in \mathbb{R} \\ \mathbb{R} & s = * \end{cases}$$

This definition implies $s \preceq *$. We extend $\mathbb{R}_*$ to vector spaces with an elementwise $+$, $\cdot$, and $\preceq$. Both operations are monotone: if $s \preceq s'$ and $t \preceq t'$, then $s + t \preceq s' + t'$ and $s \cdot t \preceq s' \cdot t'$. The proofs below rely on this property whenever they replace a quantity by its pattern inside a sum or product.

Proposition 2 (Sparsity of the augmented adjoint system)**.** Let $\rho(L, R, B, r, c) \in \mathbb{R}$ be a loss of the output of bidiagonalization, and let $(\ddot{\ddot{A}}, \ddot{v}) := \texttt{augment}(A, v)$. Let $\Lambda \in \mathbb{R}^{(N+M)\times 2K}, \Gamma \in \mathbb{R}^{2K\times 2K}, \lambda_0 \in \mathbb{R}^{N+M}, \gamma \in \mathbb{R}^{2K}$ be the adjoint variables of Proposition 1 for $\texttt{Hessenberg}_{2K}(\ddot{\ddot{A}}, \ddot{v})$ and the loss $\rho \circ \texttt{extract}$. Then, for $i \in \{1, \ldots, 2K\}$, the adjoint variables satisfy the sparsity relations

$$\begin{aligned}
&\text{For even } i: && \Lambda e_i \preceq \big[\overbrace{0\ 0\ \cdots\ 0}^{N}\ \overbrace{*\ *\ \cdots\ *}^{M}\big]^\top \\
&\text{For odd } i: && \Lambda e_i \preceq \big[\overbrace{*\ *\ \cdots\ *}^{N}\ \overbrace{0\ 0\ \cdots\ 0}^{M}\big]^\top \\
&\text{For even } i: && \Gamma^\top e_i \preceq I_{\geq} e_i \circ \big[0\ *\ 0\ *\ \cdots\ *\ 0\ *\big]^\top \\
&\text{For odd } i: && \Gamma^\top e_i \preceq I_{\geq} e_i \circ \big[*\ 0\ *\ 0\ \cdots\ 0\ *\ 0\big]^\top
\end{aligned}$$

$$\lambda_0 \preceq \big[\overbrace{0\ 0\ \cdots\ 0}^{N}\ \overbrace{*\ *\ \cdots\ *}^{M}\big]^\top \qquad \gamma \preceq \big[*\ 0\ *\ 0\ \cdots\ 0\ *\ 0\big]^\top$$

The proof at the end of this appendix is a backward induction over $i = 2K, \ldots, 1$ through the recursion of Appendix C.1.

We will give names to all nonzero entries in $\Lambda, \Gamma, \lambda_0, \gamma$. The proposition implies that Γ is lower triangular and has a checkered sparsity pattern with a zero in the lower left. The structure is best conveyed for $K = 3$, where Γ is 6×6 and its entries we name in the

following systematic manner

$$\Gamma = \begin{bmatrix} \omega_{11} & 0 & 0 & 0 & 0 & 0 \\ 0 & \sigma_{11} & 0 & 0 & 0 & 0 \\ \omega_{21} & 0 & \omega_{22} & 0 & 0 & 0 \\ 0 & \sigma_{21} & 0 & \sigma_{22} & 0 & 0 \\ \omega_{31} & 0 & \omega_{32} & 0 & \omega_{33} & 0 \\ 0 & \sigma_{31} & 0 & \sigma_{32} & 0 & \sigma_{33} \end{bmatrix} \quad \begin{matrix} \text{with} \\ \text{aliases} \end{matrix} \quad \begin{matrix} \Omega := \begin{bmatrix} \omega_{11} & 0 & 0 \\ \omega_{21} & \omega_{22} & 0 \\ \omega_{31} & \omega_{32} & \omega_{33} \end{bmatrix} \\ \\ \Sigma := \begin{bmatrix} \sigma_{11} & 0 & 0 \\ \sigma_{21} & \sigma_{22} & 0 \\ \sigma_{31} & \sigma_{32} & \sigma_{33} \end{bmatrix} \end{matrix}$$

where the definitions of Ω and Σ will turn out to be convenient later. The proposition also states that Λ has pattern:

$$\Lambda = \begin{bmatrix} \phi_1 & \mathbf{0} & \phi_2 & \mathbf{0} & \cdots & \phi_K & \mathbf{0} \\ \mathbf{0} & \xi_1 & \mathbf{0} & \xi_2 & \cdots & \mathbf{0} & \xi_K \end{bmatrix} \in \mathbb{R}^{(N+M)\times 2K} \quad \begin{matrix} \text{with} \\ \text{aliases} \end{matrix} \quad \begin{matrix} \Phi := [\phi_1 \cdots \phi_K] \\ \Xi := [\xi_1 \cdots \xi_K] \end{matrix}$$

and the entries of λ_0 and γ as

$$\lambda_0 = \begin{bmatrix} \mathbf{0} & \xi_0 \end{bmatrix}^\top \qquad \gamma = \begin{bmatrix} \eta_1 & 0 & \eta_2 & 0 & \cdots & 0 & \eta_K & 0 \end{bmatrix}^\top \quad \begin{matrix} \text{with} \\ \text{alias} \end{matrix} \quad \eta := \begin{pmatrix} \eta_1 \\ \vdots \\ \eta_K \end{pmatrix} \in \mathbb{R}^K$$

The remainder of this appendix proves Proposition 2.

D.1 Sparsity patterns of the input gradients

Because `extract` only reads entries, $\nabla_{\dddot{Q}}\rho$ and $\nabla_{\ddot{r}}\rho$ have the sparsity patterns of $\dddot{Q}$ and $\ddot{r}$, and $\nabla_{\ddot{H}}\rho$ is nonzero only on the superdiagonal, where `extract` reads $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$:

$$\nabla_{\dddot{Q}}\rho = \begin{bmatrix} \mathbf{0} & \nabla_{l_1}\rho & \mathbf{0} & \nabla_{l_2}\rho & \cdots & \mathbf{0} & \nabla_{l_K}\rho \\ \nabla_{r_1}\rho & \mathbf{0} & \nabla_{r_2}\rho & \mathbf{0} & \cdots & \nabla_{r_K}\rho & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{(N+M)\times 2K} \tag{46}$$

$$\nabla_{\ddot{r}}\rho = \begin{pmatrix} \mathbf{0} \\ \nabla_r \rho \end{pmatrix} \in \mathbb{R}^{N+M} \tag{47}$$

$$\nabla_{\ddot{H}}\rho = \begin{bmatrix} 0 & \nabla_{\alpha_1}\rho & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \nabla_{\beta_1}\rho & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \nabla_{\alpha_2}\rho & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \nabla_{\beta_2}\rho & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \ddots & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \nabla_{\beta_{K-1}}\rho & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \nabla_{\alpha_K}\rho \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \in \mathbb{R}^{2K\times 2K} \tag{48}$$

D.2 Inspecting the sparsity patterns of γ and λ_{2K}

This section proves the statements

$$\gamma \preceq \begin{bmatrix} * & 0 & * & 0 & \cdots & 0 & * & 0 \end{bmatrix}^\top$$

and

$$\lambda_{2K} \preceq \begin{bmatrix} \overbrace{0 \;\; 0 \;\; \cdots \;\; 0}^{N} & \overbrace{* \;\; * \;\; \cdots \;\; *}^{M} \end{bmatrix}^\top$$

Starting from (41), replacing K with $2K$, we plug in our sparse gradients and sparse outputs:

$$\begin{aligned} \gamma &= \nabla_{\dddot{H}} \rho e_{2K} - \dddot{Q}^\top \nabla_{\dddot{r}} \rho = \nabla_{\alpha_K} \rho e_{2K-1} - \dddot{Q}^\top \begin{pmatrix} \mathbf{0} \\ \nabla_r \rho \end{pmatrix} \\ &= \begin{pmatrix} 0 & 0 & 0 & 0 & \cdots & 0 & \nabla_{\alpha_K} \rho & 0 \end{pmatrix}^\top - \begin{bmatrix} \nabla_r \rho^\top \begin{pmatrix} r_1 & \mathbf{0} & r_2 & \mathbf{0} & \cdots & \mathbf{0} & r_K & \mathbf{0} \end{pmatrix} \end{bmatrix}^\top \\ &\preceq \begin{pmatrix} * & 0 & * & 0 & \cdots & 0 & * & 0 \end{pmatrix}^\top \end{aligned} \tag{49}$$

This pattern shows that $\gamma \in \mathbb{R}^{2K}$ is alternatingly non-zero and zero, so γ has at most K non-zero entries. We plug this pattern into (38):

$$\begin{aligned} \lambda_{2K} &= \nabla_{\dddot{r}} \rho + \dddot{Q} \gamma \\ &\preceq \begin{pmatrix} 0 & 0 & \cdots & 0 & * & * & \cdots & * \end{pmatrix}^\top + \ddot{Q} \begin{pmatrix} * & 0 & * & 0 & \cdots & 0 & * & 0 \end{pmatrix}^\top \\ &= \begin{pmatrix} 0 & 0 & \cdots & 0 & * & * & \cdots & * \end{pmatrix}^\top \end{aligned} \tag{50}$$

This pattern shows that $\lambda_{2K} \in \mathbb{R}^{N+M}$ is 0 in the leading N entries, which also held true for the input vector $\ddot{v}$.

D.3 Sparsity patterns of $\Gamma^\top$ and Λ

This section proves the remainder of the proposition, namely that

$$\text{For even } i: \quad \left(\Lambda e_i \preceq \begin{bmatrix} \overbrace{0 \;\; 0 \;\; \cdots \;\; 0}^{N} & \overbrace{* \;\; * \;\; \cdots \;\; *}^{M} \end{bmatrix}^\top \right) =: P_\Lambda(i)$$

$$\text{For odd } i: \quad \left(\Lambda e_i \preceq \begin{bmatrix} \overbrace{* \;\; * \;\; \cdots \;\; *}^{N} & \overbrace{0 \;\; 0 \;\; \cdots \;\; 0}^{M} \end{bmatrix}^\top \right) =: P_\Lambda(i)$$

$$\text{For even } i: \quad \left(\Gamma^\top e_i \preceq I_{\geq} e_i \circ \begin{bmatrix} 0 & * & 0 & * & \cdots & * & 0 & * \end{bmatrix}^\top \right) =: P_\Gamma(i)$$

$$\text{For odd } i: \quad \left(\Gamma^\top e_i \preceq I_{\geq} e_i \circ \begin{bmatrix} * & 0 & * & 0 & \cdots & 0 & * & 0 \end{bmatrix}^\top \right) =: P_\Gamma(i)$$

where we have abbreviated the propositions as $P_\Lambda(i)$ and $P_\Gamma(i)$. $P_\Lambda(2K)$ is already proven in (50). To prove the rest, we will use strong induction from $i = 2K$ down to $i = 1$, by showing that

$$P_\Lambda(i) \implies P_\Gamma(i)$$
$$\bigwedge_{j=i}^{2K} \big(P_\Lambda(j) \wedge P_\Gamma(j)\big) \implies P_\Lambda(i-1)$$

The remaining equations that solve for the unknown adjoint variables $\Gamma^\top$ and Λ are the recurrence relations (44) and (45). To prove these two implications, we will use strong induction backwards, starting from $i = 2K$ down to $i = 1$, by assuming that the propositions hold for the current and all preceding iterations $j \in \{i, i+1, \ldots, 2K\}$. The base case $i = 2K$ is already proven for even $i = 2K$ in (50). Assuming now that the hypothesis is true for some even iteration i (and all earlier iterations) we will show that the hypothesis holds for the preceding $i-1$ which is odd. To close the loop, we prove that, when the hypothesis holds for an odd i and earlier, the hypothesis will hold for the preceding $i-1$, which is even.

D.4 Showing $P_\Lambda(i) \implies P_\Gamma(i)$

We move onto equation (44) for columns of $\Gamma^\top$. We group the equation into four terms (I) - (IV), analyze each term separately, and recombine the terms at the end.

$$\Gamma_{ii}e_i + \Gamma^\top e_i = -I_{\geq} e_i \circ \Bigg[\underbrace{\dddot{Q}^\top \nabla_{\ddot{q}_i} \rho}_{\text{(I)}} + \underbrace{\dddot{Q}^\top \dddot{A}^\top \lambda_i}_{\text{(II)}}\Bigg] + \Bigg(\underbrace{\sum_{j=i-1}^{2K} h_{ij} I_{\geq} e_i \circ \nabla_{\ddot{H}} \rho e_j}_{\text{(III)}}\Bigg) - \underbrace{e_1 c \nabla_c \rho \delta_{i=1}}_{\text{(IV)}} \tag{51}$$

Expanding the sparsity patterns of the known quantities $\ddot{Q}$, $\dddot{A}$, $\nabla_{\ddot{q}_i}\rho$, and $\nabla_{\ddot{H}}\rho$ simplifies each term. For even i, the column $\ddot{q}_i$ holds $l_{i/2}$ and λ_i holds $\xi_{i/2}$; for odd i, $\ddot{q}_i$ holds $r_{(i+1)/2}$ and λ_i holds $\phi_{(i+1)/2}$.

D.4.1 Term (I)

$$\text{(I)} := \dddot{Q}^\top \nabla_{\ddot{q}_i} \rho$$

Using substitutions (29) and (46), for even i:

$$\begin{aligned}
\dddot{Q}^\top \nabla_{\ddot{q}_i} \rho &= \begin{bmatrix} \mathbf{0} & l_1 & \mathbf{0} & l_2 & \cdots & \mathbf{0} & l_K \\ r_1 & \mathbf{0} & r_2 & \mathbf{0} & \cdots & r_K & \mathbf{0} \end{bmatrix}^\top \begin{bmatrix} \nabla_{l_{i/2}} \rho \\ \mathbf{0} \end{bmatrix} \\
&= \begin{bmatrix} 0 & l_1^\top \nabla_{l_{i/2}} \rho & 0 & l_2^\top \nabla_{l_{i/2}} \rho & \cdots & 0 & l_K^\top \nabla_{l_{i/2}} \rho \end{bmatrix}^\top \\
&\preceq \begin{bmatrix} 0 & * & 0 & * & \cdots & 0 & * \end{bmatrix}^\top
\end{aligned}$$

For odd i:

$$\begin{aligned}\dddot{Q}^\top \nabla_{\ddot{q}_i} \rho &= \begin{bmatrix} \mathbf{0} & l_1 & \mathbf{0} & l_2 & \cdots & \mathbf{0} & l_K \\ r_1 & \mathbf{0} & r_2 & \mathbf{0} & \cdots & r_K & \mathbf{0} \end{bmatrix}^\top \begin{bmatrix} \mathbf{0} \\ \nabla_{r_{(i+1)/2}} \rho \end{bmatrix} \\ &= \begin{bmatrix} r_1^\top \nabla_{r_{(i+1)/2}} \rho & 0 & r_2^\top \nabla_{r_{(i+1)/2}} \rho & 0 & \cdots & r_K^\top \nabla_{r_{(i+1)/2}} \rho & 0 \end{bmatrix}^\top \\ &\preceq \begin{bmatrix} * & 0 & * & 0 & \cdots & * & 0 \end{bmatrix}^\top\end{aligned}$$

D.4.2 Term (II)

$$(\mathrm{II}) := \dddot{Q}^\top \dddot{A}^\top \lambda_i$$

Using again substitution (29) in addition to (27). We will also make use of the induction hypothesis on the sparsity of λ_i. For even i:

$$\dddot{Q}^\top \dddot{A}^\top \lambda_i = \begin{bmatrix} \mathbf{0} & l_1 & \mathbf{0} & l_2 & \cdots & \mathbf{0} & l_K \\ r_1 & \mathbf{0} & r_2 & \mathbf{0} & \cdots & r_K & \mathbf{0} \end{bmatrix}^\top \begin{bmatrix} \mathbf{0} & A \\ A^\top & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{0} \\ \xi_{i/2} \end{bmatrix} \tag{52}$$

$$= \begin{bmatrix} 0 & l_1^\top A \xi_{i/2} & 0 & l_2^\top A \xi_{i/2} & \cdots & 0 & l_K^\top A \xi_{i/2} \end{bmatrix}^\top \tag{53}$$

$$\preceq \begin{bmatrix} 0 & * & 0 & * & \cdots & 0 & * \end{bmatrix}^\top \tag{54}$$

For odd i, we get:

$$\dddot{Q}^\top \dddot{A}^\top \lambda_i = \begin{bmatrix} \mathbf{0} & l_1 & \mathbf{0} & l_2 & \cdots & \mathbf{0} & l_K \\ r_1 & \mathbf{0} & r_2 & \mathbf{0} & \cdots & r_K & \mathbf{0} \end{bmatrix}^\top \begin{bmatrix} \mathbf{0} & A \\ A^\top & \mathbf{0} \end{bmatrix} \begin{bmatrix} \phi_{(i+1)/2} \\ \mathbf{0} \end{bmatrix} \tag{55}$$

$$= \begin{bmatrix} r_1^\top A^\top \phi_{(i+1)/2} & 0 & r_2^\top A^\top \phi_{(i+1)/2} & 0 & \cdots & r_K^\top A^\top \phi_{(i+1)/2} & 0 \end{bmatrix}^\top \tag{56}$$

$$\preceq \begin{bmatrix} * & 0 & * & 0 & \cdots & * & 0 \end{bmatrix}^\top \tag{57}$$

D.4.3 Term (III)

$$(\mathrm{III}) := \sum_{j=i-1}^{2K} h_{ij} I_{\geq} e_i \circ \nabla_{\ddot{H}} \rho e_j$$

Here, we make use of the sparsity pattern of $\nabla_{\ddot{H}} \rho$ from (48), which gives that, for even i,

$$\nabla_{\ddot{H}} \rho e_i = \nabla_{\alpha_{i/2}} \rho e_{i-1}$$

and for odd i,

$$\nabla_{\ddot{H}} \rho e_i = \nabla_{\beta_{(i-1)/2}} \rho e_{i-1}$$

Column j of $\nabla_{\ddot{H}} \rho$ is thus nonzero only in row $j-1$, which the mask $I_{\geq} e_i$ removes for $j > i+1$ and keeps for $j \leq i+1$, so we can prune the sum to $j \leq i+1$ and drop the mask. Additionally, we know from (31) that $h_{ii} = 0$ for all i. Terms with indices outside

the matrices, such as $\nabla_{\beta_0}\rho$ for $i = 2$ or $h_{2K,2K+1}$ for $i = 2K$, are zero. Assuming now i even:

$$\begin{aligned}
\sum_{j=i-1}^{2K} h_{ij} I_{\geq} e_i \circ \nabla_{\ddot{H}} \rho e_j &= \sum_{j=i-1}^{i+1} h_{ij} I_{\geq} e_i \circ \nabla_{\ddot{H}} \rho e_j \\
&= h_{i,i-1} \nabla_{\ddot{H}} \rho e_{i-1} + h_{i,i} \nabla_{\ddot{H}} \rho e_i + h_{i,i+1} \nabla_{\ddot{H}} \rho e_{i+1} \\
&= h_{i,i-1} \nabla_{\ddot{H}} \rho e_{i-1} + h_{i,i+1} \nabla_{\ddot{H}} \rho e_{i+1} \\
&= \alpha_{i/2} \nabla_{\ddot{H}} \rho e_{i-1} + \beta_{i/2} \nabla_{\ddot{H}} \rho e_{i+1} \\
&= \alpha_{i/2} \nabla_{\beta_{i/2-1}} \rho e_{i-2} + \beta_{i/2} \nabla_{\beta_{i/2}} \rho e_i \\
&\preceq * \cdot e_{i-2} + * \cdot e_i
\end{aligned}$$

With a similar argument, for odd i:

$$\begin{aligned}
\sum_{j=i-1}^{2K} h_{ij} I_{\geq} e_i \circ \nabla_{\ddot{H}} \rho e_j &= h_{i,i-1} \nabla_{\ddot{H}} \rho e_{i-1} + h_{i,i+1} \nabla_{\ddot{H}} \rho e_{i+1} \\
&= \beta_{(i-1)/2} \nabla_{\ddot{H}} \rho e_{i-1} + \alpha_{(i+1)/2} \nabla_{\ddot{H}} \rho e_{i+1} \\
&= \beta_{(i-1)/2} \nabla_{\alpha_{(i-1)/2}} \rho e_{i-2} + \alpha_{(i+1)/2} \nabla_{\alpha_{(i+1)/2}} \rho e_i \\
&\preceq * \cdot e_{i-2} + * \cdot e_i
\end{aligned}$$

D.4.4 Term (IV)

$$\text{(IV)} := e_1 c \nabla_c \rho \delta_{i=1}$$

For even i, $\delta_{i=1} = 0$, and

$$e_1 c \nabla_c \rho \delta_{i=1} = \mathbf{0}$$

For odd i,

$$e_1 c \nabla_c \rho \delta_{i=1} \preceq * \cdot e_1$$

D.4.5 Combining Terms (I) - (IV)

Substituting the four terms back into (44) gives, for even i:

$$\begin{aligned}
\Gamma_{ii} e_i + \Gamma^\top e_i &= -I_{\geq} e_i \circ \Big[\text{(I)} + \text{(II)} \Big] + \text{(III)} \\
\iff \Gamma^\top e_i &= -I_{\geq} e_i \circ \Big[\text{(I)} + \text{(II)} \Big] + \text{(III)} - \Gamma_{ii} e_i \\
&\preceq I_{\geq} e_i \circ \Big[\begin{bmatrix} 0 & * & 0 & * & \cdots & 0 & * \end{bmatrix}^\top + \begin{bmatrix} 0 & * & 0 & * & \cdots & 0 & * \end{bmatrix}^\top \Big] \\
&\quad + * \cdot e_{i-2} + * \cdot e_i \\
&= I_{\geq} e_i \circ \begin{bmatrix} 0 & * & 0 & * & \cdots & 0 & * \end{bmatrix}^\top
\end{aligned}$$

The terms $* \cdot e_{i-2}$ and $* \cdot e_i$, which include $-\Gamma_{ii} e_i$, sit at even positions, so even columns of $\Gamma^\top$ are exactly zero in the odd entries. For odd i:

$$\begin{aligned}
\Gamma_{ii} e_i + \Gamma^\top e_i &= -I_{\geq} e_i \circ \Big[(\mathrm{I}) + (\mathrm{II}) \Big] + (\mathrm{III}) - (\mathrm{IV}) \\
\iff \Gamma^\top e_i &= -I_{\geq} e_i \circ \Big[(\mathrm{I}) + (\mathrm{II}) \Big] + (\mathrm{III}) - (\mathrm{IV}) - \Gamma_{ii} e_i \\
&\preceq I_{\geq} e_i \circ \Big[\begin{bmatrix} * & 0 & * & 0 & \cdots & * & 0 \end{bmatrix}^\top + \begin{bmatrix} * & 0 & * & 0 & \cdots & * & 0 \end{bmatrix}^\top \Big] \\
&\quad + * \cdot e_{i-2} + * \cdot e_i - * \cdot e_1 \\
&= I_{\geq} e_i \circ \begin{bmatrix} * & 0 & * & 0 & \cdots & * & 0 \end{bmatrix}^\top
\end{aligned}$$

Here e_{i-2}, e_i, and e_1 sit at odd positions, so the odd columns of $\Gamma^\top$ are exactly zero in the even entries. These two cases prove $P_\Lambda(i) \implies P_\Gamma(i)$.

D.5 Showing that $P_\Lambda(j)$ and $P_\Gamma(j)$ for all $j \geq i$ imply $P_\Lambda(i-1)$

For this implication, we plug the sparse outputs $\dddot{Q}$, $\dddot{H}$, and $\ddot{r}$ of $\texttt{Hessenberg}_{2K}(\dddot{A}, \ddot{v})$ into (45). We again underline the main terms with roman numerals (I) - (V), which we will investigate one by one, for even and odd i.

$$\left.\begin{matrix} -\lambda_0 & i = 1 \\ h_{i,i-1}\lambda_{i-1} & i > 1 \end{matrix}\right\} = \underbrace{\nabla_{\ddot{q}_i} \rho}_{(\mathrm{I})} + \underbrace{\dddot{A}^\top \lambda_i}_{(\mathrm{II})} - \underbrace{\sum_{j=i}^{2K} h_{ij} \lambda_j}_{(\mathrm{III})} + \underbrace{\ddot{Q}(\Gamma + \Gamma^\top) e_i}_{(\mathrm{IV})} + \underbrace{\ddot{r} \gamma^\top e_i}_{(\mathrm{V})} \tag{58}$$

We are interested in the sparsity pattern of the right-hand side, as this directly tells us about the sparsity on the left-hand side. Note that the λs on the LHS are indexed with $i-1$ and not i, which is a possible point of confusion.

D.5.1 Term (I)

By the sparsity pattern of $\nabla_{\ddot{Q}} \rho$ in (46), we directly have

$$\text{for even } i: \qquad \nabla_{\ddot{q}_i} \rho \preceq \begin{bmatrix} * & * & \cdots & * & 0 & 0 & \cdots & 0 \end{bmatrix}^\top \tag{59}$$

$$\text{for odd } i: \qquad \nabla_{\ddot{q}_i} \rho \preceq \begin{bmatrix} 0 & 0 & \cdots & 0 & * & * & \cdots & * \end{bmatrix}^\top \tag{60}$$

D.5.2 Term (II)

For even i, using the induction hypothesis:

$$\dddot{A}^\top \lambda_i = \begin{bmatrix} \mathbf{0} & A \\ A^\top & \mathbf{0} \end{bmatrix} \lambda_i \preceq \begin{bmatrix} \mathbf{0} & A \\ A^\top & \mathbf{0} \end{bmatrix} \begin{bmatrix} 0 & 0 & \cdots & 0 & * & * & \cdots & * \end{bmatrix}^\top \tag{61}$$

$$= \begin{bmatrix} * & * & \cdots & * & 0 & 0 & \cdots & 0 \end{bmatrix}^\top \tag{62}$$

For odd i, using the induction hypothesis:

$$\ddot{A}^\top \lambda_i = \begin{bmatrix} \mathbf{0} & A \\ A^\top & \mathbf{0} \end{bmatrix} \lambda_i \preceq \begin{bmatrix} \mathbf{0} & A \\ A^\top & \mathbf{0} \end{bmatrix} \begin{bmatrix} * & * & \cdots & * & 0 & 0 & \cdots & 0 \end{bmatrix}^\top \tag{63}$$

$$= \begin{bmatrix} 0 & 0 & \cdots & 0 & * & * & \cdots & * \end{bmatrix}^\top \tag{64}$$

D.5.3 Term (III)

From the sparsity of $\ddot{H}$ shown in (31), we can simplify this sum severely (all but one $h_{ij} = 0$), and use the induction hypothesis $P_\Lambda(i+1)$. For even i:

$$\sum_{j=i}^{2K} h_{ij}\lambda_j = h_{i,i+1}\lambda_{i+1} \preceq \begin{bmatrix} * & * & \cdots & * & 0 & 0 & \cdots & 0 \end{bmatrix}^\top \tag{65}$$

For odd i:

$$\sum_{j=i}^{2K} h_{ij}\lambda_j = h_{i,i+1}\lambda_{i+1} \preceq \begin{bmatrix} 0 & 0 & \cdots & 0 & * & * & \cdots & * \end{bmatrix}^\top \tag{66}$$

D.5.4 Term (IV)

Here we will use the sparsity of $\dddot{Q}$ shown in (29), and use our induction hypothesis. Since Γ is lower triangular, the ith column of $\Gamma + \Gamma^\top$ consists of $\Gamma^\top e_i$ and of the entries $(\Gamma^\top e_j)_i$ for $j > i$, so the assumption $\bigwedge_{j=i}^{2K} P_\Gamma(j)$ gives its sparsity pattern (see Example 2). For even i:

$$\ddot{Q}(\Gamma + \Gamma^\top)e_i = \begin{bmatrix} \mathbf{0} & l_1 & \mathbf{0} & l_2 & \cdots & \mathbf{0} & l_K \\ r_1 & \mathbf{0} & r_2 & \mathbf{0} & \cdots & r_K & \mathbf{0} \end{bmatrix} (\Gamma + \Gamma^\top)e_i \tag{67}$$

$$\preceq \begin{bmatrix} \mathbf{0} & * & \mathbf{0} & * & \cdots & \mathbf{0} & * \\ * & \mathbf{0} & * & \mathbf{0} & \cdots & * & \mathbf{0} \end{bmatrix} \begin{bmatrix} 0 & * & 0 & * & \cdots & 0 & * \end{bmatrix}^\top \tag{68}$$

$$= \begin{bmatrix} * & * & \cdots & * & 0 & 0 & \cdots & 0 \end{bmatrix}^\top \tag{69}$$

And, for odd i:

$$\ddot{Q}(\Gamma + \Gamma^\top)e_i = \begin{bmatrix} \mathbf{0} & l_1 & \mathbf{0} & l_2 & \cdots & \mathbf{0} & l_K \\ r_1 & \mathbf{0} & r_2 & \mathbf{0} & \cdots & r_K & \mathbf{0} \end{bmatrix} (\Gamma + \Gamma^\top)e_i \tag{70}$$

$$\preceq \begin{bmatrix} \mathbf{0} & * & \mathbf{0} & * & \cdots & \mathbf{0} & * \\ * & \mathbf{0} & * & \mathbf{0} & \cdots & * & \mathbf{0} \end{bmatrix} \begin{bmatrix} * & 0 & * & 0 & \cdots & * & 0 \end{bmatrix}^\top \tag{71}$$

$$= \begin{bmatrix} 0 & 0 & \cdots & 0 & * & * & \cdots & * \end{bmatrix}^\top \tag{72}$$

D.5.5 Term (V)

The sparsity of γ in (49) shows that $\gamma^\top e_i = 0$ for all even i, implying

$$\dddot{r}\gamma^\top e_i = \mathbf{0}$$

In the case of odd i, $\gamma^\top e_i \in \mathbb{R}$ is a scalar and, using the sparsity of $\ddot{r}$ in (30), we get

$$\dddot{r}\gamma^\top e_i \preceq \begin{bmatrix} 0 & 0 & \cdots & 0 & * & * & \cdots & * \end{bmatrix}^\top$$

D.5.6 Combining Terms (I) - (V)

Finally, the terms (I) - (V) of (58) at step i are recombined to obtain (a scaled) λ_{i-1}. For even i:

$$\left.\begin{matrix} -\lambda_0 & i=1 \\ h_{i,i-1}\lambda_{i-1} & i>1 \end{matrix}\right\} = \text{(I)} + \text{(II)} - \text{(III)} + \text{(IV)} + \text{(V)}$$
$$\preceq (1+1-1+1+0)\begin{bmatrix} * & * & \cdots & * & 0 & 0 & \cdots & 0 \end{bmatrix}^\top$$
$$\preceq \begin{bmatrix} * & * & \cdots & * & 0 & 0 & \cdots & 0 \end{bmatrix}^\top$$

And for odd i:

$$\left.\begin{matrix} -\lambda_0 & i=1 \\ h_{i,i-1}\lambda_{i-1} & i>1 \end{matrix}\right\} = \text{(I)} + \text{(II)} - \text{(III)} + \text{(IV)} + \text{(V)}$$
$$\preceq (1+1-1+1+1)\begin{bmatrix} 0 & 0 & \cdots & 0 & * & * & \cdots & * \end{bmatrix}^\top$$
$$\preceq \begin{bmatrix} 0 & 0 & \cdots & 0 & * & * & \cdots & * \end{bmatrix}^\top$$

Recall that $h_{i,i-1}$ is nonzero (see Appendix B.2). Dividing by $h_{i,i-1}$ preserves the sparsity pattern, so $P_\Lambda(i-1)$ holds. Together with the base case $P_\Lambda(2K)$ established in Appendix D.2, the backward induction is complete, and the proposition is proven. □

E Derivation of the stable adjoint system

Having established the sparsity of all adjoint variables, we now substitute the decompositions from Proposition 2 back into the Hessenberg recurrences (41), (38), (44), and (45). Each equation separates into independent N-block and M-block components due to the block-diagonal/block-off-diagonal structure of the augmented system. We treat each recurrence in turn.

E.1 Reducing the initialization

Recall from (41) that $\gamma = \nabla_{\dddot{H}}\rho\, e_{2K} - \dddot{Q}^\top \nabla_{\dddot{r}}\rho$. We have already computed in Appendix D.2 that

$$\gamma = \begin{pmatrix} 0 & 0 & \cdots & 0 & \nabla_{\alpha_K}\rho & 0 \end{pmatrix}^\top - \begin{pmatrix} r_1^\top \nabla_r \rho & 0 & r_2^\top \nabla_r \rho & 0 & \cdots & r_K^\top \nabla_r \rho & 0 \end{pmatrix}^\top$$

Extracting the odd-indexed entries gives $\eta \in \mathbb{R}^K$ with

$$\eta_n = -r_n^\top \nabla_r \rho \quad \text{for } n < K, \qquad \eta_K = \nabla_{\alpha_K} \rho - r_K^\top \nabla_r \rho \tag{73}$$

or equivalently $\eta = -R^\top \nabla_r \rho + \nabla_{\alpha_K} \rho \, e_K$. Similarly, from (38), $\lambda_{2K} = \nabla_{\dddot{r}} \rho + \dddot{Q} \gamma$. The M-block (extracting the lower half) gives

$$\xi_K = \nabla_r \rho + R\eta \tag{74}$$

where we used that $\dddot{Q}\gamma$ restricted to the M-block is $\sum_{n=1}^K r_n \eta_n = R\eta$.

E.2 Reducing the reprojection constraints

The key identity from Appendix C.1 is (40), $(\mathbf{1} - I_{\ll}) \circ \dddot{Q}^\top \Lambda = (\mathbf{1} - I_{\ll}) \circ \nabla_{\ddot{H}} \rho$, whose mask $\mathbf{1} - I_{\ll} = [\delta_{i \le j+1}]_{i,j=1}^{2K}$ keeps the upper triangle and the first subdiagonal. The odd columns of $\dddot{Q}$ hold r_n in the M-block and the even columns hold l_n in the N-block (29), and the odd columns of Λ hold ϕ_n in the N-block and the even columns hold ξ_n in the M-block (Proposition 2). Hence

$$\dddot{Q}^\top \Lambda = \begin{bmatrix} 0 & r_1^\top \xi_1 & 0 & r_1^\top \xi_2 & \cdots \\ l_1^\top \phi_1 & 0 & l_1^\top \phi_2 & 0 & \cdots \\ 0 & r_2^\top \xi_1 & 0 & r_2^\top \xi_2 & \cdots \\ l_2^\top \phi_1 & 0 & l_2^\top \phi_2 & 0 & \cdots \\ \vdots & & & & \ddots \end{bmatrix},$$

that is, $\dddot{Q}^\top \Lambda$ holds $r_n^\top \xi_m$ at position $(2n{-}1, 2m)$, $l_n^\top \phi_m$ at position $(2n, 2m{-}1)$, and zeros elsewhere. The mask $\mathbf{1} - I_{\ll}$ keeps position $(2n{-}1, 2m)$ if $2n - 1 \le 2m + 1$, that is, if $n \le m + 1$, and position $(2n, 2m{-}1)$ if $2n \le 2m$, that is, if $n \le m$. By (48), $\nabla_{\ddot{H}} \rho$ holds $\nabla_{\alpha_n} \rho$ at position $(2n{-}1, 2n)$, $\nabla_{\beta_n} \rho$ at position $(2n, 2n{+}1)$, and zeros elsewhere. Matching the kept entries gives $r_n^\top \xi_m = \delta_{nm} \nabla_{\alpha_n} \rho$ for $n \le m + 1$ and $l_n^\top \phi_m = \delta_{n+1,m} \nabla_{\beta_n} \rho$ for $n \le m$, which are the reprojection constraints

$$I_{\ge} \circ (L^\top \Phi) = \operatorname{diag}(\nabla_{\boldsymbol{\beta}} \rho, \; k{=}1) \tag{75}$$

$$I_{\ge+1} \circ (R^\top \Xi) = \operatorname{diag}(\nabla_{\boldsymbol{\alpha}} \rho) \tag{76}$$

In the $K \times K$ coordinates, the single mask $\mathbf{1} - I_{\ll}$ thus becomes $I_{\ge+1}$ for $R^\top \Xi$ and $I_{\ge}$ for $L^\top \Phi$, because the rows of r_n come one index before the rows of l_n. The zero diagonal of $L^\top \Phi$ and the zero first subdiagonal of $R^\top \Xi$ come from the zero subdiagonal of $\nabla_{\ddot{H}} \rho$, that is, from `extract` reading $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ off the superdiagonal of $\ddot{H}$ (Appendix B.2). The remaining entries of the Γ recurrence (44) build up $\Sigma + \Sigma^\top$ and $\Omega + \Omega^\top$ column by column, following the same backward loop as in the general Hessenberg case (see Example 2), but now restricted to the $K \times K$ even-even and odd-odd blocks independently.

E.3 Reducing the Λ recurrence

The recurrence (45) reads $h_{i,i-1}\lambda_{i-1} = \nabla_{\dddot{q}_i}\rho + \dddot{A}^\top\lambda_i - \sum_{j=i}^{2K} h_{ij}\lambda_j + \dddot{Q}(\Gamma+\Gamma^\top)e_i + \ddot{r}\,\gamma^\top e_i$ for $i > 1$, and $-\lambda_0 = (\cdots)$ for $i = 1$.

Even $i = 2n$. The LHS is $\alpha_n\lambda_{2n-1}$, whose nonzero part is the N-block $\alpha_n\phi_n$. Extracting the N-block of each RHS term:

$$\nabla_{\dddot{q}_{2n}}\rho \to \nabla_{l_n}\rho \qquad \dddot{A}^\top\lambda_{2n} \to A\xi_n$$
$$h_{2n,2n+1}\lambda_{2n+1} \to \beta_n\phi_{n+1} \qquad \dddot{Q}(\Gamma+\Gamma^\top)e_{2n} \to L(\Sigma+\Sigma^\top)e_n$$

The term $\ddot{r}\,\gamma^\top e_{2n} = \mathbf{0}$ since γ is zero in even entries. Moving $\alpha_n\phi_n$ to the right-hand side, using $(\Phi B^\top)e_n = \alpha_n\phi_n + \beta_n\phi_{n+1}$, and collecting all columns gives

$$0 = \nabla_L + A\Xi - \Phi B^\top + L(\Sigma+\Sigma^\top) \tag{77}$$

Odd $i = 2n-1$. The LHS is $\beta_{n-1}\lambda_{2n-2}$ (for $n > 1$) or $-\lambda_0$ (for $n = 1$), whose nonzero part is the M-block: $\beta_{n-1}\xi_{n-1}$ or $-\xi_0 = -\kappa$. Extracting the M-block of each RHS term:

$$\nabla_{\dddot{q}_{2n-1}}\rho \to \nabla_{r_n}\rho \qquad \dddot{A}^\top\lambda_{2n-1} \to A^\top\phi_n$$
$$h_{2n-1,2n}\lambda_{2n} \to \alpha_n\xi_n \qquad \dddot{Q}(\Gamma+\Gamma^\top)e_{2n-1} \to R(\Omega+\Omega^\top)e_n$$

The term $\ddot{r}\,\gamma^\top e_{2n-1} = \ddot{r}\,\eta_n$, whose M-block is res η_n (using $\ddot{r} = [\mathbf{0}; \text{res}]$). For $n > 1$, moving the left-hand side to the right combines it with $\alpha_n\xi_n$ into $(\Xi B)e_n = \beta_{n-1}\xi_{n-1} + \alpha_n\xi_n$. For $n = 1$, $(\Xi B)e_1 = \alpha_1\xi_1$, and the left-hand side $-\kappa$ moves over as $+\kappa$, which gives the term $\kappa\, e_1^\top$. Collecting all columns gives

$$0 = \nabla_R + A^\top\Phi - \Xi B + R(\Omega+\Omega^\top) + \kappa\, e_1^\top + \text{res}\,\eta^\top \tag{78}$$

E.4 Reducing the gradient formulas

From (36), $\nabla_{\dddot{A}}\rho = \Lambda\dddot{Q}^\top$, an $(N+M)\times(N+M)$ matrix. Since A enters $\dddot{A}$ twice, as the $(1,2)$-block and, transposed, as the $(2,1)$-block, $\nabla_A\rho$ is the $(1,2)$-block of $\nabla_{\dddot{A}}\rho$ plus the transpose of its $(2,1)$-block. By the alternating structure, $\Lambda\dddot{Q}^\top = \sum_{n=1}^{K}\left(\lambda_{2n-1}\,\ddot{q}_{2n-1}^\top + \lambda_{2n}\,\ddot{q}_{2n}^\top\right)$, where the odd terms $\phi_n r_n^\top$ fill the $(1,2)$-block and the even terms $\xi_n l_n^\top$ fill the $(2,1)$-block. Hence

$$\nabla_A\rho = \sum_{n=1}^{K}\phi_n r_n^\top + l_n\xi_n^\top,$$

or, in compact form,

$$\nabla_A\rho = \Phi R^\top + L\Xi^\top \tag{79}$$

From (37), $\nabla_{\ddot{v}\cdot}\rho = -c\lambda_0 = -c\begin{pmatrix}\mathbf{0}\\ \kappa\end{pmatrix}$, so extracting the M-block:

$$\nabla_v \rho = -c\kappa \tag{80}$$

From (35), $\nabla_c \rho = \ddot{v}^\top \lambda_0$. Using $\ddot{v} = \begin{pmatrix}\mathbf{0}\\ v\end{pmatrix}$ and $\lambda_0 = \begin{pmatrix}\mathbf{0}\\ \kappa\end{pmatrix}$:

$$\nabla_c \rho = v^\top \kappa = \frac{r_1^\top \kappa}{c} \qquad \Longleftrightarrow \qquad c\nabla_c \rho = r_1^\top \kappa \tag{81}$$

where we used $r_1 = cv$ from the primal constraint (3).

F Reorthonormalization and reprojection

Because Hessenberg factorization subsumes bidiagonalization for the augmented input, the bidiagonalization adjoint inherits the stability of the Hessenberg adjoint [6]. The mechanism is reprojection, which we describe next.

Let m be a mask vector with entries from $\{0, 1\}$, let L have orthonormal columns, and let q and b be vectors. Consider a constraint of the form

$$m \circ (L^\top q) = m \circ b$$

that is, within the bounds of the mask m, the vector q expressed in the basis of L equals b. A solution q of this constraint satisfies

$$q = q - L(m \circ L^\top q) + L(m \circ L^\top q) = q - L(m \circ L^\top q) + L(m \circ b).$$

Given a numerical estimate q_0 of a solution, this identity motivates the update step, repeated if necessary,

$$q_1 \leftarrow q_0 - L(m \circ L^\top q_0) + L(m \circ b),$$

which we call *reprojecting* q_0 against L. For Hessenberg factorization the relevant constraint is $Q^\top q_i = e_i$ (24), and the reprojection step followed by normalization is what we call *reorthonormalization*. For bidiagonalization, the analogous constraints are $L^\top l_i = e_i$ (4) and $R^\top r_i = e_i$ (5). Our forward pass applies the reorthonormalization step twice per iteration, the "twice is enough" rule of Golub and Van Loan [4]. In the backward pass, the reprojection constraints (13)–(14) take the form $m \circ (L^\top q) = m \circ b$ with known right-hand sides, and our implementation applies the reprojection step once per iteration to each new adjoint vector ϕ_n and ξ_n. Figure 5 shows that both corrections are necessary in single precision. Without reorthonormalization, the forward pass loses the reconstruction $LBR^\top \approx A$, with an error of 0.4 already at $n = 8$, while the reorthonormalized forward pass stays below 10^{-7} (left). Without reprojection, the error of the adjoint grows from $5 \cdot 10^{-4}$ at $n = 32$ to $2 \cdot 10^{8}$ at $n = 120$, while the reprojection-corrected adjoint stays below $1.5 \cdot 10^{-6}$ (right).

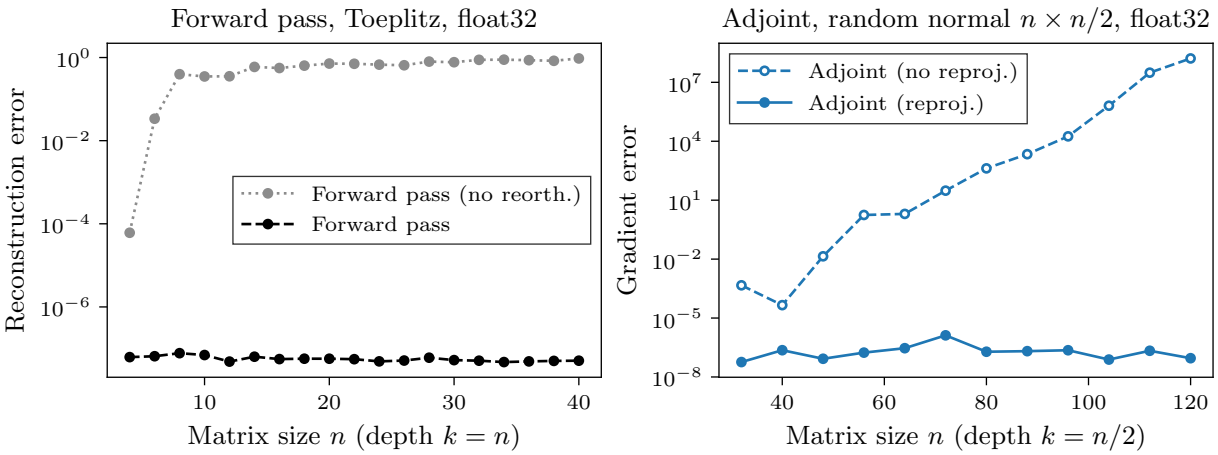


Figure 5: Left: forward reconstruction error $\|A - LBR^\top\|_{\text{rms}}$ with and without reorthonormalization, on symmetric Toeplitz matrices $C_{ij} = 1/(1 + |i - j|)$ with $k = n$ in single precision. Right: gradient error $\|\partial(LBR^\top)/\partial A - I\|_{\text{rms}}$ of the adjoint with and without the reprojection step, on dense random normal $n \times (n/2)$ matrices with $k = n/2$ in single precision.

G Alternative derivations (possibly dropped from the final paper)

For completeness, we record the classic forward constraints of bidiagonalization and the associated adjoint system written out column by column. These constraints are equivalent to the stable system of Theorem 1 but are stated directly in the original coordinates rather than derived through the augmented Hessenberg detour.

G.1 Columnwise expanded unstable adjoint

Define $b_n\lambda_{n+1} := \mathbf{0}$ and $b_0\rho_0 := \mathbf{0}$

$$0 = \nabla l_n + a_n\lambda_n + b_n\lambda_{n+1} - A\rho_n + l_n\sigma_n \tag{82}$$
$$0 = \nabla r_n - A^\top\lambda_n + b_{n-1}\rho_{n-1} + a_n\rho_n + r_n\omega_n + \delta_{n,1}\kappa \tag{83}$$
$$\rho_K = -\nabla\text{res} \tag{84}$$
$$\nabla a_n + l_n^\top\lambda_n + r_n^\top\rho_n = 0 \text{ for } n \in \{1,\dots,K\} \tag{85}$$
$$\nabla b_n + l_n^\top\lambda_{n+1} + r_{n+1}^\top\rho_n = 0 \text{ for } n \in \{1,\dots,K-1\} \tag{86}$$
$$\nabla c - \tilde{r}^\top\kappa = 0 \tag{87}$$
$$\nabla\tilde{r} = -c\kappa \tag{88}$$
$$\nabla A = \sum_{n=1}^{K} -\lambda_n r_n^\top - l_n\rho_n^\top \tag{89}$$

G.2 Unstable primal constraints

Define $l_0 b_0 := 0$ and $\text{res} := r_{K+1}b_K$

$$\begin{aligned}
r_1 &= c\tilde{r} \\
\alpha_n l_n &= Ar_n - \beta_{n-1}l_{n-1} \\
\beta_n r_{n+1} &= A^\top l_n - \alpha_n r_n \\
\langle l_i, l_i\rangle &= 1 \text{ for } i \in \{1,\dots,K\} \\
\langle r_i, r_i\rangle &= 1 \text{ for } i \in \{1,\dots,K\}
\end{aligned}$$

Symbolically, one can show by induction that these constraints, if satisfied, imply $\langle l_i, l_j\rangle = \delta_{i,j}$ and $\langle r_i, r_j\rangle = \delta_{i,j}$. In practice however, we need to reorthonormalize the produced vectors during iteration. These classic forward constraints give rise to the adjoint system below.

G.3 Unstable adjoint system

This system can be derived from the bidiagonalization forward constraints directly; the columnwise form appears in Appendix G.1.

$$\rho_K = -\nabla \mathrm{res} \tag{90}$$
$$c\nabla c = r_1^\top \kappa \tag{91}$$
$$\nabla \tilde{r} = -c\kappa \tag{92}$$
$$0 = \nabla L - AP + \Lambda B^\top + L\Sigma \tag{93}$$
$$0 = \nabla R - A^\top \Lambda + PB + R\Omega + \delta_{n,1}\kappa \tag{94}$$
$$\langle l_n, \lambda_n \rangle + \langle r_n, \rho_n \rangle = \nabla a_n \text{ for } n \in \{1, \dots, K\} \tag{95}$$
$$\langle l_n, \lambda_{n+1} \rangle + \langle r_{n+1}, \rho_n \rangle = \nabla b_n \text{ for } n \in \{1, \dots, K-1\} \tag{96}$$
$$\nabla A = \sum_{n=1}^{K} -\lambda_n r_n^\top - l_n \rho_n^\top \tag{97}$$

H Experimental setup

All experiments ran on the CPU of an Apple M1 Pro with 16 GB of memory, using JAX 0.11.2. Every experiment script stores raw measurements in the `results` directory of the code, from which `plot_figures.py` draws all figures and tables.

H.1 Full depth

Whenever forward reorthonormalization is enabled, our implementation applies two reorthonormalization passes per Krylov step, the "twice is enough" rule of Golub and Van Loan [4]. Autodiff differentiates through whatever the forward pass does. The adjoint always runs the reorthonormalized forward pass; "no reproj." turns off the reprojection step in the backward pass of the adjoint. Gradient accuracy is measured by bidiagonalizing A with $k = \min(N, M)$ steps from a standard normal start vector, reconstructing $LBR^\top$, and reporting the RMS error $\|\partial(LBR^\top)/\partial A - I\|_{\mathrm{rms}}$ of the Jacobian, which we compute by reverse-mode differentiation of the reconstruction. Figure 1 uses dense random normal $n \times n$ matrices in double precision (left) and symmetric Toeplitz matrices $C_{ij} = 1/(1 + |i - j|)$ in single precision (right; symmetric positive-definite, with $\kappa(C) \approx 15$ at $n = 40$). Figure 5 uses the same Toeplitz matrices for the forward reconstruction error $\|A - LBR^\top\|_{\mathrm{rms}}$ (left) and dense random normal $n \times (n/2)$ matrices with $k = n/2$ in single precision (right). For Figure 2, the loss is a fixed random linear combination of all outputs (L, R, B, r, c) in single precision. Compile time is the wall-clock time of lowering and compiling the forward pass or the vector-Jacobian product of the forward pass with `jax.jit`, for a diagonal 950×950 operator. Run time is the median over five runs after a warm-up run, for dense random normal $n \times n$ matrices with $k = n$; autodiff was not

run where XLA's buffer assignment for the value and gradient exceeded 8 GB, half of the machine's memory.

Further results. In double precision, autodiff through the reorthonormalized iteration and the adjoint, with or without reprojection, reproduce the identity Jacobian to within $2 \cdot 10^{-15}$ (Figure 1, left), and the error of autodiff without reorthonormalization grows from 10^{-15} at $n = 4$ to 10^{-3} at $n = 16$. In single precision (right), the error of the adjoint without reprojection exceeds 1 from $n = 20$ on (Appendix F). The NaNs of autodiff through the reorthonormalized iteration are specific to full depth: the residual after the last step vanishes in exact arithmetic and here is exactly zero in single precision, and the derivative of the Euclidean norm at zero is undefined. One step fewer, $k = n - 1$, gives finite gradients. Without reorthonormalization, autodiff avoids the zero but inherits the loss of orthogonality of the forward pass, with errors of order one.

At $n = 1170$, the backward pass of the adjoint takes 3.4 s and the forward pass 3.7 s, and the adjoint needs 66 MB, where autodiff would need 58 GB (Figure 2). Up to $n = 370$, the two backward passes are equally fast; at $n = 570$, autodiff takes seven times as long as the adjoint (3.0 s versus 0.43 s). Compile time does not grow with the depth for either method, because both methods run the iteration as a compiled loop: for a diagonal 950×950 operator and k from 20 to 100, compile time stays between 0.08 and 0.11 s for the forward pass and between 0.13 and 0.17 s for either backward pass.

H.2 Partial depth

All partial-depth experiments evaluate the loss (18) in single precision (float32) and differentiate the loss with four methods: the adjoint of Theorem 1; autodiff through the reorthonormalized iteration; the Lanczos adjoint on $A^\top A$, which writes the loss as $\rho = \|v\|^2 e_1^\top \log(T)\, e_1$, with the tridiagonal matrix T of k Lanczos steps on $A^\top A$, and differentiates the loss with the reorthogonalized Lanczos adjoint of Krämer et al. [6]; and augmented Arnoldi, which computes $\overline{\texttt{Bidiagonalize}}_k$ of Appendix B.2 with $2k$ Arnoldi steps on the augmented matrix and differentiates the result with the Arnoldi adjoint of Krämer et al. [6]. The bidiagonalization-based methods evaluate (18) through the singular value decomposition of B; the Lanczos adjoint evaluates $\|v\|^2 e_1^\top \log(T)\, e_1$ through the eigendecomposition of T. For augmented Arnoldi, we read α_i and β_i off the averaged super- and subdiagonals of the $2k \times 2k$ Hessenberg matrix. The Lanczos adjoint and augmented Arnoldi call `tridiag_sym` and `hessenberg` from `matfree` 0.6.2, the library that accompanies Krämer et al. [6], with full reorthogonalization and custom gradients; `matfree` implements reorthogonalized Lanczos through the `matfree` Arnoldi code.

Accuracy (Figure 3). We draw $A = U \operatorname{diag}(s) V^\top \in \mathbb{R}^{1000 \times 500}$ with U and V from the QR decompositions of standard normal matrices and $s_i = \kappa^{-(i-1)/499}$, round A to float32, and draw a standard normal start vector v; five seeds per configuration. The reference is the double-precision adjoint applied to the rounded matrix, so the reported

errors measure arithmetic errors rather than the rounding of A. In double precision, the three other methods agree with the reference to within $1.3 \cdot 10^{-12}$ (autodiff), $3.2 \cdot 10^{-11}$ (augmented Arnoldi), and $7.9 \cdot 10^{-8}$ (Lanczos on $A^\top A$, at $\kappa(A) = 10^6$). We report $\|g - g_{\text{ref}}\|/\|g_{\text{ref}}\|$ for the gradient g with respect to all entries of A. The left panel uses $k = 250$ and $\kappa(A) \in \{10^1, 10^{1.5}, \ldots, 10^6\}$; the right panel uses $\kappa(A) = 10^4$ and $k \in \{25, 50, 100, 150, 200, 250, 300\}$. The Lanczos adjoint on $A^\top A$ returns NaNs for two of five matrices at $\kappa(A) = 10^5$ and three of five at $\kappa(A) = 10^6$, because rounding errors in products with $A^\top A$ make some Ritz values negative. The right panel of Figure 3 shows the same mechanism as a function of depth: deeper Krylov spaces resolve smaller singular values, whose squares are lost in the rounding errors of $A^\top A$. At $\kappa(A) = 10^4$, the Lanczos error grows from $2 \cdot 10^{-4}$ at $k = 25$ to 69% at $k = 300$, while the bidiagonalization-based methods stay below $2 \cdot 10^{-4}$. At partial depth, the last residual does not vanish, and autodiff through the reorthonormalized iteration is as accurate as the adjoint, as at full depth in double precision; Krämer et al. [6, Table 2] observe the same for the Arnoldi iteration.

Memory and run time (Figure 4). The operator is $A(\theta)v = \theta * v$ (full discrete convolution) with $\theta = e_{16} + 0.1\,\xi$ for standard normal $\xi \in \mathbb{R}^{31}$, and v is standard normal. Memory is the sum of argument, output, and temporary buffer sizes that XLA assigns to the compiled value-and-gradient program. Because this number is available after compilation, we also report the memory for programs that would not fit into memory. Run times are medians over three runs after a warm-up run; programs above 12 GB were not run. Before running the experiments, all four gradient methods agreed to within $2.4 \cdot 10^{-6}$ on a smaller instance ($n = 2000$, $k = 16$). The memory of autodiff is 12 GB at $k = 128$, 48 GB at $k = 256$, and 190 GB at $k = 512$, more than an 80 GB accelerator holds.

SuiteSparse matrices (Table 1). We use the matrices `Kemelmacher`, `tomographic1`, `image_interp`, `ESOC`, and `Rucci1` from the SuiteSparse collection [2, 5], scale the nonzero values of each matrix to $\max_{ij} |a_{ij}| = 1$, and round the scaled values to float32. Products with A and $A^\top$ are segment sums over the nonzeros, and we differentiate with respect to the vector of nonzero values. The start vector is $v = A^\top u$ for a standard normal u, treated as a constant; this start vector keeps the Krylov space inside the row space of A and thereby avoids zero Ritz values for rank-deficient matrices. The reference is the double-precision adjoint on the float32 matrix. Memory is measured as for Figure 4. Programs above 8 GB, half of the machine's memory, were not run; run times are medians over three runs after a warm-up run, or a single run for programs that take longer than five seconds. The matrices come from imaging and least-squares applications and have up to 7.8 million nonzeros. Autodiff would need between 6 GB and 277 GB, so we ran autodiff only on the smallest matrix; on the largest, `Rucci1`, autodiff would need more than three times the memory of an 80 GB accelerator. The adjoint needs about 1.5× the memory and 2× the run time of the forward pass, with gradient errors between $1.4 \cdot 10^{-6}$ and $6.3 \cdot 10^{-4}$. On `Rucci1`, augmented Arnoldi needs 11.8 GB. The gradients of the Lanczos adjoint on $A^\top A$ reach 1.4% error on `ESOC`; on two of the five matrices, the Lanczos adjoint is as accurate

		Bidiagonalization		Lanczos on $A^\top A$		
Operator	$(N+M)/M$	total (s)	orthog. (s)	total (s)	orthog. (s)	orthog. ratio
convolution	2.0	0.40	0.25	0.16	0.14	1.8
`Kemelmacher`	3.9	0.28	0.16	0.09	0.03	4.6
`tomographic1`	2.2	1.23	0.73	0.64	0.35	2.1
`image_interp`	3.0	2.76	1.87	1.11	0.81	2.3
`ESOC`	9.6	5.46	2.11	2.78	0.18	12.0
`Rucci1`	19.0	18.87	12.43	4.39	0.74	16.8

Table 2: CPU time in seconds of the k-step forward pass of bidiagonalization and of the Lanczos iteration on $A^\top A$, both with full reorthogonalization ("total"), and the part of the total spent orthogonalizing ("orthog."), measured as the difference to the run time without reorthogonalization. The last column is the ratio of the two orthogonalization times.

as the adjoint.

Where the run time goes (Table 2). To compare the work of bidiagonalization and of the Lanczos iteration on $A^\top A$, we timed the k-step forward passes of both, with and without full reorthogonalization, on the convolution operator of Figure 4 ($k = 128$) and on the matrices of Table 1 ($k = 100$). The difference between the two run times is the time spent orthogonalizing. Without reorthogonalization, `matfree` runs the three-term Lanczos recurrence. Per step, bidiagonalization orthogonalizes against L and R, with N and M rows, whereas the Lanczos iteration on $A^\top A$ orthogonalizes against one basis with M rows. The ratio of the orthogonalization times follows the ratio $(N + M)/M$ of the basis sizes. The time without reorthogonalization, mostly matrix-vector products, also differs between the two methods but by less. The Lanczos iteration on $A^\top A$ orthogonalizes only against R, because the Lanczos iteration applies $A^\top A$ without forming L in between; skipping L saves this work but also squares the condition number. On `Rucci1`, where $(N + M)/M = 19$, orthogonalization takes 12.4 s of our 18.9 s forward pass but only 0.7 s of the 4.4 s Lanczos forward pass.